

A Neural Mobility Operator for Many-Body Rigid-Particle Dynamics in Stokes Flow

A. Author

Affiliation

Abstract

Particulate flows at low Reynolds number are ubiquitous in biological and engineering systems, but accurate Stokesian-dynamics simulations remain expensive because they must resolve long-ranged hydrodynamic coupling together with near-contact lubrication and many-body effects that shape suspension dynamics. We introduce NeMO, a Neural Mobility Operator for large-scale Stokes flow simulations at the fast, approximate end of the accuracy–cost spectrum. Rather than learning the mobility operator as a monolithic black box, NeMO decomposes it into an analytical self term, modular learned near-field corrections, and a GPU-resident far-field solver. The learned components have fixed, interpretable roles, capturing two-body self/lubrication corrections, two-body cross interactions, and many-body neighborhood effects, while long-range interactions are handled with a GPU-optimized LBVH treecode. This separation allows the expensive learned components to focus on local hydrodynamic physics, where simple pairwise approximations are least accurate, while the smoother far-field response is treated by a fast classical accelerator. The implementation is explicitly throughput-oriented, exposing fine-grained parallelism and using arithmetically intensive kernels to exploit modern GPU hardware. Across accuracy tests, canonical validation problems, and large sedimentation simulations, NeMO captures substantially richer near-field dynamics than simple pairwise mobility operators at only modest additional cost. On modern high-end GPU hardware, NeMO scales to simulations with tens of millions of particles, reaches millions of particle updates per second, and delivers substantial speedups over recent machine-learning-based baselines.

Keywords: Stokesian Dynamics, Machine Learning, Mobility Operator

1. Introduction

Particulate flows at low Reynolds number are ubiquitous in nature and engineering applications, ranging from biological systems such as blood flow to industrial processes such as wastewater treatment and the sedimentation of suspensions. Accurate simulation of these flows requires resolving the many-body hydrodynamic interactions induced by the surrounding viscous fluid. These interactions may be represented either through the grand mobility operator, which maps external forces and torques to translational and angular velocities, or through the corresponding resistance operator, which maps particle velocities to forces and torques. In either formulation, the computation is expensive: hydrodynamic interactions are long-ranged, owing to slowly decaying Stokes Green’s functions, while rigid particles impose no-slip constraints on their surfaces. As a result, each particle is coupled to all others, and near-field interactions are complicated by close-range singularities, making it challenging to retain physical fidelity without incurring rapidly increasing computational cost as the system size grows.

Existing methods span a broad accuracy-cost spectrum. At the high-fidelity end are boundary-resolved approaches such as boundary element method (BEM) and Method of Fundamental Solutions (MFS), where surface velocities or tractions are discretized and solved directly [1, 2, 3]. These can be very accurate, but become prohibitively expensive for large suspensions [4]. Moving down the accuracy-cost spectrum, Stokesian Dynamics (SD) methods form a broad family of intermediate-resolution approaches [5], combining multipole approximations for longer-ranged interactions with lubrication corrections at short range. At the fast approximate end are pairwise kernel summations based on Rotne-Prager-Yamakawa (RPY) tensors [6, 7, 8]. RPY is explicit and non-iterative for mobility application (block

matrix-vector products only), which makes it attractive for very large, dilute suspensions where this level of physics is often sufficient.

In this paper, we introduce **NeMO** (**N**eural **M**obility **O**perator), designed to accelerate large-scale Stokesian dynamics on modern parallel hardware like GPU. The goal is to incorporate richer near-field physics while keeping computational cost close to simple pairwise RPY-style baselines. Central to NeMO is a set of neural-network blocks trained on high-fidelity mobility data generated by expensive solvers. Each of these neural blocks is designed to capture a specific physical dynamic and together form a comprehensive surrogate for the full mobility operator. This design is also motivated by hardware: the core RPY kernel has very low arithmetic intensity (≈ 0.18 FLOPs/byte), so GPU performance is usually memory-bandwidth bound. On modern GPUs, this leaves substantial compute headroom, which NeMO utilizes to add richer physics at modest runtime cost.

In recent years, there has been growing interest in using machine learning, and neural networks in particular, to approximate PDE solution operators. This includes both Eulerian field-based architectures, such as Fourier Neural Operators, and Lagrangian or graph-based variants designed for particle systems [9, 10, 11]. In principle, these models provide a flexible framework for learning maps between function spaces from data. However, a generic data-driven operator model does not automatically enforce or exploit the special structure of the Stokesian Dynamics mobility problem. For a fixed particle configuration, the grand mobility is a force-linear operator mapping generalized forces and torques to translational and angular velocities; it also satisfies important symmetry, reciprocity, and energy-dissipation properties, while near-contact interactions exhibit singular lubrication behavior. These considerations motivate learning architectures that are tailored to the particulate Stokes flow, rather than treating the mobility operator as an unconstrained black-box map. Several recent works have explored this direction. For example, HIGNN [12] exploits the linearity of the mobility operator by learning self-, two-body, and three-body interaction contributions from simulation data, in a spirit similar to the many-body-expansion ideas used in molecular simulation [13]. Another work [14] has explored similar decompositions, learning self and three-body hydrodynamic components from data.

Despite these advances, several practical gaps remain. First, existing learned mobility models often do not fully exploit the structure of the grand mobility operator, particularly the reciprocal symmetry implied by Stokes flow. Enforcing this symmetry improves physical consistency and model efficiency by reducing the number of independent quantities that must be learned, enabling more compact and data-efficient surrogate models [15].

Second, the tradeoffs of learned operators remain under-discussed. Any fast mobility approximation must balance cost, accuracy, generality, and robustness. ML-based methods add another layer to this tradeoff, since their errors may reflect not only approximation choices but also systematic biases inherited from the training data or model design. These issues are especially important for particle simulations, where changes in volume fraction, geometry, forcing, or near-contact statistics can shift the distribution encountered during deployment.

A final limitation concerns the treatment of long-range hydrodynamic interactions. While classical fast summation methods are well developed, their integration with learned near-field mobility kernels remains underexplored from both accuracy and computational-cost perspectives. This is especially important in non-periodic domains, where fast Fourier transform methods cannot be readily applied and sedimentation-like dynamics can produce highly non-uniform particle distributions. In such regimes, FMM-like accelerations encounter load-balancing or GPU-efficiency bottlenecks [16].

In NeMO, we exploit the linearity of the Stokes equations to construct the mobility operator from a set of modular components, each assigned a fixed and interpretable role. The near-field contribution is represented by three neural-network blocks that use the local particle geometry to model two-body lubrication effects, two-body pairwise interactions, and many-body neighborhood corrections to the pairwise response. These blocks are built from physics-informed, coordinate-invariant features, which restrict the hypothesis space and enforce the reciprocal block structure of the grand mobility operator by construction. We train the components in stages so that their responsibilities remain separated rather than overlapping. This modular design improves interpretability and allows individual components to be enabled, disabled, or replaced to tune the cost-accuracy tradeoff for a given simulation target.

As in classical Stokesian Dynamics, NeMO is built around a near-field/far-field decomposition, a strategy that dates back to the original SD formulation and continues to underlie modern accelerated methods [5, 17, 18, 19]. The framework is agnostic to the choice of long-range solver: its learned and modular near-field components can be coupled to existing FMM-, Ewald-, or treecode-style far-field methods. For fully GPU-resident simulations, however, we introduce a bounding volume hierarchy (BVH) based treecode far-field accelerator, which keeps the computation on device and avoids the bottlenecks associated with CPU-oriented long-range solvers. This provides a more practical

path for large-scale, non-periodic simulations with strongly non-uniform particle distributions.

We validate NeMO on a broad set of problems, spanning static and dynamic test cases, small canonical configurations and large particle systems, using both well-established benchmarks from the literature and new experiments designed for this work. We use targeted diagnostic experiments to expose systematic biases in the learned components, helping identify where the framework is most reliable and where its practical limitations emerge. Alongside qualitative comparisons, we report quantitative error metrics against high-fidelity reference solutions.

NeMO’s design is strongly guided by modern parallel hardware, allowing it to incorporate richer learned near-field computations while keeping their runtime overhead relative to simpler pairwise approximations such as RPY modest. Combined with a fully GPU-resident far-field accelerator, this translates into strong end-to-end efficiency at scale. In large-scale sedimentation experiments, NeMO substantially outperforms a recent ML-based mobility baseline [12], even while using fewer GPU resources.

2. Methodology

2.1. Background: Mobility Operator

In this article, we focus on the motion of rigid particles immersed in Stokes flow. At low Reynolds number, inertial effects are negligible compared with viscous effects, and the incompressible Navier–Stokes equations reduce to the steady Stokes equations,

$$-\nabla p + \mu \nabla^2 \mathbf{u} + \mathbf{f} = 0, \quad (1)$$

$$\nabla \cdot \mathbf{u} = 0. \quad (2)$$

Here, $\mathbf{u}(\mathbf{x})$ is the fluid velocity, $p(\mathbf{x})$ is the pressure, μ is the dynamic viscosity, and $\mathbf{f}(\mathbf{x})$ denotes an applied body-force density or forcing term. The first equation expresses local force balance between pressure gradients, viscous stresses, and applied forcing, while the second enforces incompressibility. Together, these equations capture the target physics of low-Reynolds-number particulate flow: particle motion is dominated by viscous hydrodynamic interactions, with no inertial memory, so the fluid responds instantaneously and linearly to applied forces and torques.

This linearity is central to mobility-based descriptions of rigid-particle suspensions. For rigid particles, the no-slip boundary condition requires the fluid velocity on the surface of particle i to match a rigid-body velocity,

$$\mathbf{u}(\mathbf{x}) = \mathbf{V}_i + \boldsymbol{\Omega}_i \times (\mathbf{x} - \mathbf{X}_i), \quad \mathbf{x} \in \partial B_i, \quad (3)$$

where $\mathbf{X}_i$ is the particle center, $\mathbf{V}_i$ is its translational velocity, and $\boldsymbol{\Omega}_i$ is its angular velocity. Because the Stokes equations are linear, the velocity induced by a collection of particles can be written as a superposition of disturbance flows generated by the forces and torques applied to all particles. This leads to the grand mobility relation

$$\mathbf{U} = \mathbf{M}(\mathbf{X})\mathcal{F}, \quad (4)$$

Here, $\mathcal{F} \in \mathbb{R}^{6N}$ collects the external forces and torques on the N particles, $\mathbf{U} \in \mathbb{R}^{6N}$ collects the corresponding translational and angular velocities at the particle centers, and $\mathbf{M}(\mathbf{X}) \in \mathbb{R}^{6N \times 6N}$ is the configuration-dependent grand mobility operator, restricted here to force–torque inputs and rigid-body velocity outputs. The configuration $\mathbf{X}$ contains the particle positions and, when needed, their orientations. More general Stokesian-dynamics formulations may also include stresslets and higher force moments, together with the associated rate-of-strain constraints. These additional moment equations are not solved explicitly here; instead, their effects are represented only through the reduced mobility data used for training and validation, and through the learned near-field corrections.

Each off-diagonal 6×6 block $\mathbf{M}_{ij}$ describes the velocity response of target particle i to forces and torques applied to source particle j . In general, this response is not purely pairwise: it depends not only on the relative configuration of particles i and j , but also on the surrounding particles. Similarly, the diagonal blocks $\mathbf{M}_{ii}$ describe the self-mobility of particle i , which can also be modified by nearby particles. The operator $\mathbf{M}$ is dense because hydrodynamic interactions in Stokes flow are long-ranged. It is also symmetric positive semidefinite, as implied by the Lorentz reciprocal theorem and the non-negativity of viscous dissipation.

Although M maps forces linearly to velocities, its dependence on the particle configuration X is highly nonlinear. This motivates a many-body expansion of the configuration-dependent mobility, a perspective widely used in molecular simulation and recent learned mobility models [13, 20, 12, 21]:

$$\mathbf{U}_i \approx \mathbf{U}_i^{(1)} + \mathbf{U}_i^{(2)} + \mathbf{U}_i^{(3)} + \dots, \quad (5)$$

with

$$\mathbf{U}_i^{(1)} = \mathbf{m}^{(1)}(\mathbf{X}_i) \mathcal{F}_i, \quad \mathbf{U}_i^{(2)} = \sum_{j \neq i} \mathbf{m}^{(2)}(\mathbf{X}_i, \mathbf{X}_j) \mathcal{F}_{ij}, \quad \mathbf{U}_i^{(3)} = \sum_{j \neq i} \sum_{k \neq i, j} \mathbf{m}^{(3)}(\mathbf{X}_i, \mathbf{X}_j, \mathbf{X}_k) \mathcal{F}_{jk},$$

and similarly for higher-order terms. Here, $\mathbf{U}_i^{(1)}$, $\mathbf{U}_i^{(2)}$, and $\mathbf{U}_i^{(3)}$ denote the self, pairwise, and three-body contributions to the rigid-body velocity of particle i , respectively. The generalized force $\mathcal{F}_{ij} \in \mathbb{R}^{12}$ denotes the concatenated force and torque vectors for particles i and j . The kernels $\mathbf{m}^{(n)}$ are learnable modules, often implemented with neural networks that capture how the local geometric configuration modulates each n -body contribution. In practice, one truncates this hierarchy, and includes only the leading terms to balance physical fidelity and computational cost.

Notes. $\text{DiagBlock}(\cdot)$ and $\text{Block}(\cdot)$ denote reconstruction of the 6×6 mobility blocks from the predicted scalar coefficients using the analytic tensor bases in Eqs. (17) and (19). For the diagonal correction $\mathbf{m}_s^{(2)}$, the translation–rotation blocks use opposite signs so that the reconstructed 6×6 block is symmetric. The routine TreecodeRPY evaluates the far-field contribution with the LBVH/Barnes–Hut traversal and excludes near-field interactions to avoid double counting.

2.2. Design Overview

We now use the many-body expansion in Eq. (5) as a starting point to derive the operator decomposition used in NeMO. Rather than treating the expansion as a literal sequence of separate one-body, two-body, three-body, and higher-body models, we reorganize it into a small number of terms that correspond directly to block structures of the grand mobility matrix. This gives each component of the architecture a clear responsibility and makes the later model definitions more explicit.

Starting from Eq. (5), we first collect all three-body and higher-order contributions into a single many-body correction term. Thus, for particle i ,

$$\mathbf{U}_i \approx \mathbf{U}_i^{(1)} + \mathbf{U}_i^{(2)} + \mathbf{U}_i^{(n)}, \quad (6)$$

where $\mathbf{U}_i^{(1)}$ is the isolated-particle self-interaction, $\mathbf{U}_i^{(2)}$ contains pairwise two-body interactions, and $\mathbf{U}_i^{(n)}$ represents the collective correction due to three-body and higher-order hydrodynamic effects.

Next, we further decompose the two-body and many-body terms according to which particle carries the applied generalized force. Let $\mathcal{F}_{ij} = [\mathcal{F}_i, \mathcal{F}_j]$, and $\mathcal{F}_i = [\mathbf{F}_i^T, \mathbf{T}_i^T]^T$ denote the external force and torque applied to particle i . With this distinction, the two-body contribution can be written as

$$\mathbf{U}_i^{(2)} = \underbrace{\sum_{j \neq i} \mathbf{m}_s^{(2)}(\mathbf{X}_i, \mathbf{X}_j) \mathcal{F}_i}_{\mathbf{U}_{i,s}^{(2)}} + \underbrace{\sum_{j \neq i} \mathbf{m}_t^{(2)}(\mathbf{X}_i, \mathbf{X}_j) \mathcal{F}_j}_{\mathbf{U}_{i,t}^{(2)}}. \quad (7)$$

Here, the subscripts s and t distinguish the two ways in which a pair interaction enters the mobility operator. The self term, $\mathbf{m}_s^{(2)}$, maps the generalized force and torque applied to particle i back to the velocity response of the same particle, while accounting for the hydrodynamic modification induced by the presence of particle j . In this sense, $\mathbf{m}_s^{(2)}$ is a two-body correction to the diagonal mobility block M_{ii} . The transfer term, $\mathbf{m}_t^{(2)}$, maps the generalized force and torque applied to source particle j to the velocity response of target particle i , and therefore contributes to the off-diagonal mobility block M_{ij} .

We apply the same distinction to the many-body correction:

$$\mathbf{U}_i^{(n)} = \underbrace{\mathbf{m}_s^{(n)}(\mathbf{X}_i, \mathcal{N}_i) \mathcal{F}_i}_{\mathbf{U}_{i,s}^{(n)}} + \underbrace{\sum_{j \neq i} \mathbf{m}_t^{(n)}(\mathbf{X}_i, \mathbf{X}_j, \mathcal{N}_{ij}) \mathcal{F}_j}_{\mathbf{U}_{i,t}^{(n)}}. \quad (8)$$

Algorithm 1 NeMO workflow (offline training and online mobility application)

Offline stage (done once)

Require: High-fidelity MFS solver, cutoff radius r_c , maximum neighborhood size $K_{\max}$

- 1: Generate two-particle training samples (i, j) with random separations
- 2: Train $m_s^{(2)}$ and $m_t^{(2)}$ on pair data using the analytic self term $m^{(1)}$
- 3: Freeze $m_s^{(2)}$ and $m_t^{(2)}$
- 4: Generate local many-body samples $(i, j, \mathcal{N}_{ij})$
- 5: Train $m_t^{(n)}$ on the residual after subtracting $m^{(1)}$ and the two-body prediction

Online stage (one mobility application)

Require: Particle configuration $X = \{x_i\}_{i=1}^N$, applied forces/torques $F = \{F_i\}_{i=1}^N$

Ensure: Rigid-body velocities $U = \{U_i\}_{i=1}^N$

- 6: Build near-field neighbor list

$$\mathcal{N}_i = \{j \neq i : \|x_i - x_j\| \leq r_c\}$$

- 7: Build LBVH tree and far-field cluster moments for all particles

- 8: **for** $i = 1$ to N **do**

- 9: $U_i \leftarrow m^{(1)}(x_i) F_i$

▸ analytic self-mobility

- 10: **for all** $j \in \mathcal{N}_i$ **do**

- 11: Compute pair features

$$\phi_{ij} = (d_{ij}, d_{ij}^2, d_{ij}^3, h_{ij})$$

- 12: $\alpha_{ij}^{(s)} \leftarrow m_s^{(2)}(\phi_{ij})$

- 13: $U_i \leftarrow U_i + \text{DiagBlock}(\alpha_{ij}^{(s)}, \hat{r}_{ij}) F_i$

▸ adds the j th correction to M_{ii}

- 14: **end for**

- 15: **for all** $j \in \mathcal{N}_i$ **do**

- 16: Construct local line-neighborhood descriptor ψ_{ij} from the $K_{\max}$ closest particles to the midpoint of the pair (i, j)

- 17: $\alpha_{ij}^{(t)} \leftarrow m_t^{(2)}(\phi_{ij})$

- 18: $\alpha_{ij}^{(n)} \leftarrow m_t^{(n)}(\phi_{ij}, \psi_{ij})$

- 19: $M_{ij}^{nf} \leftarrow \text{Block}(\alpha_{ij}^{(t)} + \alpha_{ij}^{(n)}, \hat{r}_{ij})$

- 20: $U_i \leftarrow U_i + M_{ij}^{nf} F_j$

- 21: **end for**

- 22: $U_i \leftarrow U_i + \text{TreecodeRPY}(i, X, F, r_c)$

▸ far-field only; interactions inside r_c are excluded

- 23: **end for**

- 24: **return** U
-

Here, $\mathcal{N}_i$ denotes the local neighborhood around particle i , while $\mathcal{N}_{ij}$ denotes the local neighborhood around the line connecting the centers of target i and source j . Combining these pieces gives the five-term decomposition

$$\mathbf{U}_i \approx \mathbf{U}_i^{(1)} + \mathbf{U}_{i,s}^{(2)} + \mathbf{U}_{i,t}^{(2)} + \mathbf{U}_{i,s}^{(n)} + \mathbf{U}_{i,t}^{(n)}. \quad (9)$$

Equivalently, grouping the terms by their mobility-matrix block structure,

$$\begin{aligned} \mathbf{U}_i \approx & \underbrace{\left[\mathbf{m}^{(1)}(\mathbf{X}_i) + \sum_{j \neq i} \mathbf{m}_s^{(2)}(\mathbf{X}_i, \mathbf{X}_j) + \mathbf{m}_s^{(n)}(\mathbf{X}_i, \mathcal{N}_i) \right]}_{\substack{\text{diagonal block} \\ M_{ii} \text{ contribution}}} \mathcal{F}_i \\ & + \underbrace{\sum_{j \neq i} \left[\mathbf{m}_t^{(2)}(\mathbf{X}_i, \mathbf{X}_j) + \mathbf{m}_t^{(n)}(\mathbf{X}_i, \mathbf{X}_j, \mathcal{N}_{ij}) \right]}_{\substack{\text{off-diagonal block} \\ M_{ij}, j \neq i}} \mathcal{F}_j. \end{aligned} \quad (10)$$

This decomposition is the organizing principle behind the NeMO architecture. The isolated self-interaction is evaluated analytically, while the remaining near-field corrections are represented by modular learned components. The two-body self-correction modifies the diagonal mobility block, the two-body cross term provides the pairwise off-diagonal baseline, and the retained many-body cross correction accounts for local collective effects beyond isolated pairs. We omit the many-body self term, $\mathbf{m}_s^{(n)}$, because after the isolated and pairwise self contributions have been separated out, the leading irreducible many-body self correction is higher order, scaling as $O(r^{-7})$ in the far field. In the remainder of this section, we describe the retained components individually and then explain how the slowly decaying far-field contribution is handled by a fast summation method.

Self-interaction: The self-interaction term represents the mobility of an isolated rigid particle. Rather than learning this contribution, we evaluate it analytically from classical Stokes-flow results. In general, the self-mobility maps the force $\mathbf{F}_i$ and torque $\mathbf{T}_i$ acting on particle i to its translational and angular velocities, $\mathbf{V}_i^{(1)}$ and $\mathbf{\Omega}_i^{(1)}$, through the 6×6 block relation

$$\begin{bmatrix} \mathbf{V}_i^{(1)} \\ \mathbf{\Omega}_i^{(1)} \end{bmatrix} = \mathbf{m}^{(1)} \begin{bmatrix} \mathbf{F}_i \\ \mathbf{T}_i \end{bmatrix} = \begin{bmatrix} \mathbf{M}_{VF,ii}^{(1)} & \mathbf{M}_{VT,ii}^{(1)} \\ \mathbf{M}_{\Omega F,ii}^{(1)} & \mathbf{M}_{\Omega T,ii}^{(1)} \end{bmatrix} \begin{bmatrix} \mathbf{F}_i \\ \mathbf{T}_i \end{bmatrix}. \quad (11)$$

For a spherical particle of radius a suspended in a fluid of viscosity μ , this self-mobility is isotropic, and the translation-rotation coupling blocks vanish. The isolated-particle mobility therefore reduces to

$$\begin{bmatrix} \mathbf{M}_{VF,ii}^{(1)} & \mathbf{M}_{VT,ii}^{(1)} \\ \mathbf{M}_{\Omega F,ii}^{(1)} & \mathbf{M}_{\Omega T,ii}^{(1)} \end{bmatrix} = \begin{bmatrix} \frac{1}{6\pi\mu a} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \frac{1}{8\pi\mu a^3} \mathbf{I} \end{bmatrix}, \quad (12)$$

so that

$$\mathbf{V}_i^{(1)} = \frac{1}{6\pi\mu a} \mathbf{F}_i, \quad \mathbf{\Omega}_i^{(1)} = \frac{1}{8\pi\mu a^3} \mathbf{T}_i. \quad (13)$$

Similar analytic expressions exist for ellipsoidal particles [22]. While being more involved, they are still relatively cheap to evaluate. Rest of the near-field terms are all machine learnable modules, and are designed to capture the physics of multi-particle interactions that are not accounted for by the self-mobility, as we discuss next.

Two-Body Self-Correction, $\mathbf{m}_s^{(2)}$: This model provides a correction to the previous self-interaction term by incorporating the effect of neighboring particles in a pairwise fashion. For a generalized force and torque applied to particle i , this two-body term captures how the presence of a nearby particle j modifies both the translational and angular velocity responses of particle i . In particular, the learned block includes the neighbor-mediated translation-rotation coupling between the force-torque and velocity components [23]. This is crucial for accurately modeling near-field hydrodynamic interactions, which are dominated by lubrication effects when particles are in close proximity.

Coupled with the self-interaction term, the overall diagonal block of grand mobility matrix $\mathbf{M}$ can thus be written as:

$$M_{ii} = \mathbf{m}^{(1)}(x_i) + \sum_{j \neq i} m_s^{(2)}(x_i, x_j). \quad (14)$$

This correction decays rapidly with inter-particle distance, scaling as $O(r^{-4})$, and is therefore primarily a near-field contribution. In practice, it can be truncated beyond a short cutoff without introducing significant error. Conceptually, $m_s^{(2)}$ plays a role similar to pairwise lubrication corrections: it adjusts the single-particle mobility to account for the resistance induced by a nearby neighbor. For spherical particles, analytical lubrication expressions are available in the asymptotically close regime, but these formulas are reliable only at very small gaps, e.g., $r < 2.01a$, and require specialized treatment for "mid-field" [24]. More importantly, neither the learned self-correction $m_s^{(2)}$ nor standard analytical lubrication corrections act as a collision-handling mechanism; additional constraints or repulsive/contact models are still needed to prevent true particle overlap, which is beyond the scope of this current work [25].

Two-body Cross Term, $m_t^{(2)}$: This model provides a baseline for the off-diagonal blocks M_{ij} of the grand mobility matrix, mapping the force applied to a source particle j to the velocity response of a target particle i . This mapping depends only on the pair configuration, namely the positions of the source and target particles, and their orientations for ellipsoids. A key property of this term is that it decays much more slowly than the other terms, scaling as $O(r^{-1})$, which gives rise to the long-range and dense nature of the mobility operator. An analytical alternative to $m_t^{(2)}$ is the Rotne-Prager-Yamakawa (RPY) tensor, which is effective in the far field but less accurate at short separations, where finite-size and near-field hydrodynamic effects become important.

N-body Correction Term, $m_t^{(n)}$: This component is responsible for providing a correction to the previous two-body cross-interaction term by incorporating the effect of neighboring particles. For any force applied to a source particle j , this many-body term captures how the presence of nearby particles modifies the velocity response of particle i to that force. Together with the two-body cross term, the overall off-diagonal block of grand mobility matrix M can be written as:

$$M_{ij} = m_t^{(2)}(X_i, X_j) + m_t^{(n)}(X_i, X_j, \mathcal{N}_{ij}), \quad k \in \mathcal{N}_{ij}, k \neq i, k \neq j \quad (15)$$

Here, $m_t^{(2)}$ is the baseline two-body cross-interaction from source j to target i , while $m_t^{(n)}$ adds the correction induced by neighboring particles in the local set $\mathcal{N}_{ij}$ (excluding i and j). Note that this many-body off-diagonal correction term decays at a rate of $O(r^{-3})$.

This term is the most challenging component to model, since it must represent the collective effect of several nearby particles rather than an isolated pair interaction. It is therefore also the dominant source of NeMO's modeling error. One possible alternative, used in [12] and [14], is to further decompose this contribution into separate three-body corrections, one for each neighboring particle k , and then sum the resulting velocity corrections. This makes the learning problem simpler and reduces the need for hand-designed many-body features, but it also increases the evaluation cost, since the neural network must be evaluated separately for each neighbor. In our experiments, a single many-body correction model achieves comparable accuracy while avoiding this additional per-neighbor inference cost, as we demonstrate in the experiments section (see Fig. 3).

Far-field: Of the last 3 components discussed, only the two-body cross term $m_t^{(2)}$ has a slow decay rate of $O(r^{-1})$, which means that it is the dominant contribution to the far-field interactions. The other two terms, $m_s^{(2)}$ and $m_t^{(n)}$, decay much faster and are negligible for distant particles.

While the $m_t^{(2)}$ term is long-range, the physics beyond a certain cutoff distance is relatively smooth and can be captured by simpler analytical kernels. For example, in our experiments, even a relatively deep, 5-layer neural network fails to provide any substantial improvement over the analytical Rotne-Prager-Yamakawa (RPY) tensor baseline after $6a$ particle pair-distance. Although non-spherical particles, such as ellipsoids, may push this empirical cutoff to a higher value, for sufficiently large simulations there is always a threshold beyond which improvement from learnable kernels become negligible [26]. Moreover, beyond $20a$, the point particle approximation (Oseen tensor) is sufficient to capture the physics with negligible error.

This means all neural network-based components can be restricted to near-field. While the decay rate of these terms are different, for simplicity, we use a single cutoff r_c for all terms. This distance cutoff r_c splits the grand mobility into near- and far-field parts:

$$\mathbf{M}(\mathbf{X}) = \mathbf{M}_{nf}(\mathbf{X}; r_c) + \mathbf{M}_{ff}(\mathbf{X}; r_c). \quad (16)$$

Here, the nearfield operator $\mathbf{M}_{nf}$ is now a sparse operator, but the physics it models is relatively complex. Whereas the far-field operator $\mathbf{M}_{ff}$ is dense, the physics it represents is smoother and can be handled by standard fast kernel-sum methods, such as FMM, treecode, or particle-mesh/Ewald-type [27, 19]. This near-field/far-field decomposition is a common strategy in classical Stokesian dynamics methods [5], and has remained central in subsequent acceleration work [17, 18].

Nevertheless, we found that existing fast-summation methods did not naturally match NeMO’s need for a GPU-accelerated approximation that remains efficient in the moderate-accuracy, highly non-uniform regimes targeted here. For this reason, we implement a Barnes–Hut treecode built on top of a linear bounding volume hierarchy (LBVH). The design and implementation of the treecode are described in the next subsection.

Summary of the design: The resulting architecture is deliberately explicit and non-iterative. Once the neural components are trained offline, an online mobility application consists only of evaluating analytical terms, learned local corrections, and the far-field summation, followed by accumulating the corresponding velocity contributions. No additional constraint, resistance, or surface-traction system is solved during time stepping. In this sense, the computational pattern and cost structure of NeMO are closer to fast mobility-application methods, such as pairwise RPY summation, than higher-fidelity formulations that solve for boundary densities, stresslets, or other constrained hydrodynamic degrees of freedom.

The decomposition of M in Eq. (10) is useful in two related but distinct ways. First, it makes the method modular: the isolated self term, two-body corrections, many-body correction, and far-field solver can be trained, evaluated, modified, or replaced as separate components. Second, the particular way in which we decompose M gives the components interpretable roles, several terms have analogues in classical mobility approximations. NeMO assigns compact learned models only to the parts of the mobility response that are both local and difficult to capture analytically, while retaining analytical structure and fast classical solvers where they remain effective. The learned models are thus embedded as small components within a physics-aware computational pipeline, rather than replacing the full mobility operator.

This separation is especially important for the two-body contribution. By splitting the two-body interactions into the rapidly decaying self-correction ($m_s^{(2)}$) and the slowly decaying cross term ($m_t^{(2)}$), NeMO can restrict the expensive learned evaluations to the near field while treating the long-ranged part of ($m_t^{(2)}$) with standard fast-summation methods, such as treecode, FMM, or Ewald-type solvers. This compatibility is critical to the performance of the method, as the far-field component dominates the runtime in a large-scale system.

2.3. Neural Components

2.3.1. Design

In this section, we describe the design of each of the neural components: $m_s^{(2)}$, $m_t^{(2)}$ and $m_t^{(n)}$, which share certain structural similarities. Each neural component maps the local geometry around a target particle to a 6×6 block within the grand mobility operator (“kernel” from now on), which in turn linearly maps forces and torques on either the source or target particle to the velocity response of the target particle. It is quite possible to use a regular multilayer perceptron (MLP) to learn the nonlinear mapping from geometry to 6×6 kernel, but doing so would ignore the underlying symmetries and structure of the mobility operator. Instead, we leverage well-known theoretical results about the structure of the mobility kernels to design a physically-informed network that makes grand mobility operator symmetric by construction.

For spheres (or any axisymmetric ellipsoidal particle in general), an off-diagonal transfer kernel M_{ij} , with $i \neq j$, has the form [28]

$$M_{ij} = \begin{bmatrix} A_1 I + B_1 \hat{r}_{ij} \hat{r}_{ij}^T & C_1 [\varepsilon_{abc} \hat{r}_{ij,c}]_{ab} \\ C_1 [\varepsilon_{abc} \hat{r}_{ij,c}]_{ab} & A_2 I + B_2 \hat{r}_{ij} \hat{r}_{ij}^T \end{bmatrix}, \quad (17)$$

where I is the 3×3 identity matrix, $\hat{r}_{ij}$ is the unit vector from particle j to particle i , and ε_{abc} is the Levi-Civita symbol. Define the skew-symmetric matrix

$$E(\hat{r}_{ij}) = [\varepsilon_{abc} \hat{r}_{ij,c}]_{ab}, \quad E(\hat{r}_{ij})^T = -E(\hat{r}_{ij}). \quad (18)$$

Intuitively, the block structure expresses the velocity response in terms of longitudinal and transverse contributions relative to the separation vector. The scalar functions A_1 , B_1 , A_2 , and B_2 determine these responses, while C_1 governs

the translation–rotation coupling. This reduces the problem from predicting 36 entries per block to learning only five scalar functions, substantially lowering the model complexity.

The two-body self-correction $m_s^{(2)}$ also retains all five scalar functions, including a generally nonzero translation–rotation coefficient. Because this term contributes to the diagonal block M_{ii} , its lower-left coupling block is the transpose of its upper-right coupling block. Its reconstruction therefore uses

$$m_s^{(2)}(X_i, X_j) = \begin{bmatrix} A_{s,1}I + B_{s,1} \hat{r}_{ij} \hat{r}_{ij}^T & C_s E(\hat{r}_{ij}) \\ -C_s E(\hat{r}_{ij}) & A_{s,2}I + B_{s,2} \hat{r}_{ij} \hat{r}_{ij}^T \end{bmatrix}. \quad (19)$$

Since $E(\hat{r}_{ij})$ is skew-symmetric, the opposite signs make the full 6×6 self-correction block symmetric. The coefficient C_s captures translation induced by torque and rotation induced by force on particle i through the hydrodynamic influence of particle j . For the two-body cross term $m_t^{(2)}$, all five scalar functions A_1, B_1, C_1, A_2, B_2 depend on the inter-particle distance r_{ij} . For the many-body correction term $m_t^{(n)}$, the off-diagonal block structure in Eq. (17) is retained, but the scalar functions also depend on the local neighborhood configuration.

Encoding Symmetry: We enforce the reciprocal symmetry of the off-diagonal mobility blocks, $\mathbf{M}_{ij} = \mathbf{M}_{ji}^\top$, by ensuring the scalar coefficients (i.e. A_1, B_1, C_1, A_2, B_2 in Eq. (17)) are rotation-invariant. The directional dependence is then introduced analytically through tensor bases constructed from $\hat{r}_{ij}$ and the Levi–Civita symbol. For the diagonal self-correction, the sign convention in Eq. (19) separately enforces $m_s^{(2)} = m_s^{(2)T}$. In this way, the networks learn only scalar functions, while the required tensorial structure is recovered analytically.

To preserve this rotation-invariance property, the network inputs are built only from relative configuration information. These features include inter-particle distances, surface gaps, and orientation-dependent quantities that are invariant under rigid translations and relabeling of identical particles. Using the same scalar functions for every pair then gives $\mathbf{M}_{ij} = \mathbf{M}_{ji}^\top$ by construction. This scalar-coefficient/tensor-basis construction follows the same general idea used in modern machine-learned interatomic models and equivariant neural networks, where known physical symmetries are built into the model rather than learned from data [29].

Neural network architecture: We use a fully connected feedforward network to map feature vectors derived from local particle geometry to the corresponding scalar coefficients. Because the model is constrained by hydrodynamic structure and symmetry (Eq. (17)), the network only needs to learn a low-dimensional set of scalar functions, enabling smaller architectures than prior learned mobility designs [12, 14]. We intentionally keep the models narrow so that weights and activations fit in fast on-chip GPU memory, reducing memory-bandwidth pressure during inference [30]. For the two-body models, we use a 4-layer network with three hidden layers of sizes 64, 32, and 64. For the many-body correction term $m_t^{(n)}$, we use a deeper 6-layer network with hidden layers alternating between widths of 64 and 128, allowing it to capture the richer structure of multi-particle interactions.

The two-body networks are extremely lightweight, requiring only about 5×10^3 multiply-add operations, or roughly 10^4 floating-point operations, per forward pass. The deeper many-body correction network remains modest in cost at about 4×10^4 multiply-adds, or 8×10^4 floating-point operations.

Why neural networks? Predicting the mobility kernel is, in principle, a supervised multivariate regression problem, for which many machine-learning approaches could be used. We use neural networks here for three practical reasons.

First, the model must be trained end-to-end through a fixed physics-based post-processing step. The network predicts a small set of scalar mobility coefficients, which are then mapped analytically to rigid-body velocities. Because both stages are differentiable, we can backpropagate a loss on the final velocities through the full pipeline. This is natural for neural networks, but less convenient for standard non-differentiable regressors such as decision trees or random forests.

Second, neural networks are expressive function approximators that can capture complex nonlinear relationships in high-dimensional spaces. Some terms, such as the self-interaction contribution, could likely be modeled with simpler regressors. But neural networks provide a single framework that can be used for all interaction types, including the more difficult many-body corrections.

Third, the usual concerns about neural networks are partly mitigated in this setting. Interpretability comes mainly from the operator decomposition and staged training procedure, which assign each network a fixed physical role, rather than from the internal weights of the MLP itself. Computational cost is also controlled by keeping the architectures small, so that inference remains inexpensive on highly parallel GPU hardware.

We experimented with adding residual connections and dense connectivity patterns, similar to ResNet and DenseNet architectures, but did not observe any systematic improvement in accuracy for the scalar functions of interest. Given the added complexity and memory footprint of these architectures, we retain the simpler multilayer perceptron design.

2.3.2. Input features

The scalar functions learned by the neural networks depend on the local geometric configuration of the particles. We encode this configuration through a small set of invariant input features.

For the two-body models, the encoding is straightforward. Let $d_{ij} = \|\mathbf{r}_{ij}\|$ denote the center-to-center distance between particles i and j . Before forming the polynomial distance features, we center this distance using the mean distance in the two-body training set. Specifically, if $\bar{d}_{\text{train}}$ denotes the training-set mean of d_{ij} , we define the centered distance as $\tilde{d}_{ij} = d_{ij} - \bar{d}_{\text{train}}$. The polynomial features are then formed from this centered quantity, giving $\tilde{d}_{ij}$, $\tilde{d}_{ij}^2$, and $\tilde{d}_{ij}^3$.

We also include the minimum surface-to-surface separation $h_{ij} = d_{ij} - (a_i + a_j)$, where a_i and a_j are the effective radii along the center-to-center direction. Thus, the final pair feature vector is

$$\phi_{ij} = (\tilde{d}_{ij}, \tilde{d}_{ij}^2, \tilde{d}_{ij}^3, h_{ij}).$$

The same value of $\bar{d}_{\text{train}}$ is used at inference time, so that the input normalization is fixed after training. For non-spherical, axisymmetric particles, relative orientations must also be encoded through rotation-invariant descriptors; we leave the details of that extension to future work.

For the off-diagonal many-body correction, the network input combines pair-level information for the target-source pair (i, j) with a local geometric description of nearby particles around that pair. The pair-level features are the same as those used in the two-body cross-interaction model. To account for local many-body effects, we additionally construct a pair-symmetric descriptor for particles near the line segment connecting i and j .

For each target-source pair (i, j) , let $\mathbf{r}_{ij} = \mathbf{x}_i - \mathbf{x}_j$, $\ell = \|\mathbf{r}_{ij}\|$, and $\hat{\mathbf{z}} = \mathbf{r}_{ij}/\ell$, where $\mathbf{r}_{ij}$ points from the source particle j to the target particle i . We select up to K_{max} neighboring particles, excluding i and j , closest to the pair midpoint $\mathbf{m}_{ij} = (\mathbf{x}_i + \mathbf{x}_j)/2$ within a prescribed cutoff radius. When fewer than K_{max} valid neighbors are present, the remaining slots are padded with zero feature vectors and marked as invalid using a binary mask.

For each valid neighbor k , we define $\mathbf{r}_{ki} = \mathbf{x}_k - \mathbf{x}_i$ and $\mathbf{r}_{kj} = \mathbf{x}_k - \mathbf{x}_j$, with magnitudes $r_{ki} = \|\mathbf{r}_{ki}\|$ and $r_{kj} = \|\mathbf{r}_{kj}\|$. The first three features are the pair-symmetric distance combinations

$$r_{\text{sum}} = r_{ki} + r_{kj}, \quad r_{\text{diff}} = |r_{ki} - r_{kj}|, \quad r_{\text{prod}} = r_{ki}r_{kj}.$$

These quantities describe how close the neighbor is to the two particles without assigning a preferred role to either one.

We also express the position of k in the local coordinate frame of the pair. Let $\mathbf{v}_k = \mathbf{x}_k - \mathbf{m}_{ij}$, and define the signed axial coordinate $u = \mathbf{v}_k \cdot \hat{\mathbf{z}}$ and radial distance $\rho = \|\mathbf{v}_k - u\hat{\mathbf{z}}\|$. To make the descriptor even under interchange of the target and source labels, we use

$$\frac{|u|}{\ell}, \quad \left(\frac{u}{\ell}\right)^2, \quad \frac{\rho}{\ell}.$$

These normalized quantities indicate whether the neighbor lies near one end of the pair, near the midpoint, or away from the pair axis.

To give the network additional sensitivity to close configurations, we include the symmetric inverse-distance features

$$\text{inv}_{\text{sum}} = \frac{1}{r_{ki}} + \frac{1}{r_{kj}}, \quad \text{inv}_{\text{prod}} = \frac{1}{r_{ki}r_{kj}}, \quad \text{inv}_{\text{absdiff}} = \left| \frac{1}{r_{ki}} - \frac{1}{r_{kj}} \right|.$$

Finally, we include the angle at the neighbor k formed by the two segments connecting k to i and j , encoded by

$$\cos \theta_k = \frac{(\mathbf{x}_i - \mathbf{x}_k) \cdot (\mathbf{x}_j - \mathbf{x}_k)}{r_{ki}r_{kj}}, \quad \cos \theta_k \in [-1, 1].$$

Thus, each neighbor contributes the 10-dimensional descriptor

$$\left(r_{\text{sum}}, r_{\text{diff}}, r_{\text{prod}}, \frac{|u|}{\ell}, \left(\frac{u}{\ell} \right)^2, \frac{\rho}{\ell}, \text{inv}_{\text{sum}}, \text{inv}_{\text{prod}}, \text{inv}_{\text{absdiff}}, \cos \theta_k \right).$$

The final input to the many-body correction network is obtained by concatenating the pair-level features, the K_{max} neighbor descriptors, and the corresponding binary mask. For $K_{\text{max}} = 10$, the neighbor descriptors contribute 100 entries and the mask contributes 10 entries. Together with the four pair-level features used in the two-body cross model, this gives a 114-dimensional input vector.

2.4. Far Field

2.4.1. Compatibility with fast solvers

The slow-decaying nature of the $m_i^{(2)}$ component leads essentially to the following pairwise sum:

$$\mathbf{u}_{i,ff} = \sum_{\substack{j=1 \\ j \neq i}}^N m_i^{(2)}(\mathbf{x}_i, \mathbf{x}_j) F_j \quad (20)$$

The goal of the far-field component is to efficiently evaluate this sum for all particles, which is a classic N -body problem. Direct evaluation scales as $O(N^2)$ and becomes prohibitively expensive for large systems. Several well-established fast-summation methods address this class of problem, including the Fast Multipole Method (FMM) [19] and Ewald-type methods. A key goal of our modular M_{nf} design was to ensure that it can interface well with existing approaches. To verify this compatibility, we validated our implementation against STKFMM [31] and PVFMM [32], using both RPY and Stokeslet kernels.

2.4.2. Choosing a fast solver

However, NeMO’s far-field stage imposes a more specific set of performance constraints than those typically targeted by general-purpose fast-summation packages. At the large particle counts where moderate-fidelity mobility operators are most useful, the long-range evaluation can dominate the cost of a mobility application. For example, in the sedimentation benchmark reported in Sec. 3.4, a GPU-accelerated barycentric FMM implementation [33] was about an order of magnitude slower than the near-field stage, which would largely erase the benefit of the fast learned near-field models. Because NeMO applies the mobility to a changing particle configuration at each time step, unlike boundary-based or MFS-based solves that can amortize hierarchy construction and upward-pass costs over multiple operator evaluations for a fixed geometry, these stages must themselves be inexpensive. The method must also remain efficient for the strongly non-uniform particle configurations produced by sedimentation dynamics. Finally, the target accuracy is moderate: the far-field error only needs to be small relative to the learned near-field modeling error. This pushes the fast-summation method toward low-order approximations, where there is less arithmetic work per accepted interaction to amortize traversal, scheduling, and memory-access overheads, making efficient GPU parallelization more difficult than in higher-order, high-accuracy regimes [34].

2.4.3. Treecodes

To address these constraints, we use a Treecode [27] built on top of a Linear Bounding Volume Hierarchy (LBVH). As in FMM, the method first builds a spatial hierarchy and then computes source moments in an upward pass. Evaluation is performed independently for each target particle: each target traverses the source tree, accepting distant clusters and descending to child nodes otherwise for nearby ones. This avoids several components of a full FMM pipeline, such as interaction lists, multipole-to-local (M2L) translations, and a downward pass. The resulting method has a higher asymptotic cost than an optimized FMM, with the usual $O(N \log N)$ -type scaling rather than $O(N)$, but the design is much simpler and exposes fine-grained parallelism that is useful on GPUs [34, 35].

To partition the source space, we use a linear bounding volume hierarchy (LBVH), rather than the octree structure more commonly used in FMM implementations. LBVH is a more natural fit for non-uniform particle distributions. Unlike an octree, which recursively subdivides space, an LBVH organizes the particles themselves into a binary

hierarchy. The resulting rectangular bounding volumes can adapt more naturally to clustered or elongated particle distributions. Another practical advantage of this approach is that the LBVH is a widely used primitive in ray tracing, meaning highly efficient tree construction algorithms for GPU already exist [36]. This makes it feasible to rebuild the source tree and recompute its moments at every time step.

In the upward pass, we use a second-order Taylor expansion, retaining monopole, dipole, and quadrupole moments for each cluster. During evaluation, traversal is controlled by a standard multipole acceptance criterion (MAC): a node is accepted when its size-to-distance ratio is below a prescribed threshold, and otherwise the traversal descends to its children. The MAC parameter therefore gives a direct speed–accuracy knob. We keep the expansion order low so that each accepted-cluster evaluation remains cheap enough for a simple target-wise traversal kernel. This increases the number of M2P and P2P evaluations, but is a reasonable tradeoff in the moderate-accuracy regime targeted here [34]. The flexible traversal also makes it straightforward to use different kernels at different stages of traversal: direct particle-to-particle interactions near the cutoff are evaluated with the RPY kernel, while accepted far-field clusters are evaluated using a lower-order Stokeslet-based M2P approximation, similar in spirit to Kernel-Aggregated FMM [31].

The resulting far-field contribution is thus:

$$\mathbf{u}_{i,ff} = \sum_{C \in \mathcal{A}(i)} \mathcal{T}_{\text{M2P}}(\mathbf{x}_i; \mathbf{c}_C, \mathbf{M}_C) + \sum_{j \in \mathcal{D}(i)} G_{\text{RPY}}(\mathbf{x}_i, \mathbf{x}_j) \mathbf{F}_j. \quad (21)$$

Here, $\mathcal{A}(i)$ is the set of MAC-accepted far-field clusters for target i , while $\mathcal{D}(i)$ contains the particles reached by descending to opened leaves. The M2P operator $\mathcal{T}_{\text{M2P}}$ evaluates the second-order Stokeslet expansion about the cluster center $\mathbf{c}_C$ using the stored moments $\mathbf{M}_C$. Particles in $\mathcal{D}(i)$ are instead evaluated directly with the RPY kernel $G_{\text{RPY}}(\mathbf{x}_i, \mathbf{x}_j)$. To avoid double counting with M_{nf} , accepted clusters must lie outside the near-field cutoff region; when traversal reaches a leaf, individual pairs with $r_{ij} < r_c$ are skipped because they are already included in the learned near-field operator.

2.5. Dataset Generation & Training

To obtain high-quality training data, we use the Method of Fundamental Solutions (MFS), following [3], to solve the Stokes equations for rigid particles under applied forces and torques. MFS is a boundary method in which the fluid velocity is represented as a linear combination of fundamental solutions (stokeslets and rotlets) placed at fictitious source points just inside each particle. Enforcing the no-slip boundary condition at a set of collocation points on the particle surfaces yields a linear system for the unknown strengths of the fundamental solutions. Solving this system provides an accurate approximation of the fluid velocity field and, consequently, of the rigid-body velocities of the particles.

For two-body interactions, each training sample consists of two spheres. We place the target particle at the origin and sample the source particle within a spherical shell around it. For the many-body correction term, we start from the same source–target pair and construct a local candidate neighborhood around it. Specifically, we define a spherical shell centered at the midpoint of the line segment connecting the source and target particles. The shell is chosen large enough to contain both particles, together with additional surrounding volume in which nearby particles may influence the source–target mobility. We then sample $2K$ non-overlapping candidate particles from this shell. Since this initial shell can include particles that are geometrically far from the source–target interaction and therefore weakly relevant to the many-body correction, we rank the candidates using the symmetric scalar metric

$$q_k = r_{ki} r_{kj}, \quad r_{ki} = \|\mathbf{x}_k - \mathbf{x}_i\|, \quad r_{kj} = \|\mathbf{x}_k - \mathbf{x}_j\|.$$

The K particles with the smallest values of q_k are retained as the local neighborhood N_{ij} , giving the $K + 2$ -particle configuration used in the MFS solve. This selection favors particles that are close to either member of the pair, or lie in the region between them, and is symmetric with respect to exchanging source and target. Geometrically, level sets of q_k form Cassini-oval-like surfaces, with the source and target particles acting as the two foci. The resulting $K + 2$ particle configuration is then passed to MFS. For every training sample, a randomly oriented unit force and unit torque are applied only to the source particle; the target and all neighboring particles are force- and torque-free (note that source and target are the same particle for $m_s^{(2)}$ training dataset). MFS is then used to compute the induced rigid-body velocity of the target particle, which serves as the training target for the regression task.

We train the learned components in Eq. 10 in stages rather than optimizing them jointly. We first train the two-body models, $m_s^{(2)}$ and $m_t^{(2)}$, on pair configurations. After these models are converged, we freeze their weights and train the many-body correction $m_t^{(n)}$ only on the residual that remains after subtracting the analytic self term and the predicted two-body cross contributions from the MFS target. Because nearby particles give rise to more complex hydrodynamic interactions, the datasets for the two-body and many-body cases are generated with a slight bias toward closer configurations. Samples with overlapping particles are rejected.

This staged procedure is important for interpretability. The input to $m_t^{(n)}$ contains the same pairwise descriptors used by the two-body models, together with additional neighborhood features, so its feature space is effectively a superset of the two-body feature space. If all components were trained at once, the many-body network could absorb pairwise physics that should belong to $m_s^{(2)}$, producing a decomposition with similar aggregate error but weaker modularity and less interpretable blocks. By freezing the lower-order models first and forcing $m_t^{(n)}$ to predict only the remaining residual, each network retains a clear physical role within the many-body expansion.

Model evaluation: For a collection of N reference rigid-body velocities $\{\mathbf{u}_i^{\text{true}}\}_{i=1}^N$ and corresponding model predictions $\{\mathbf{u}_i^{\text{pred}}\}_{i=1}^N$. Let

$$\delta \mathbf{u}_i = \mathbf{u}_i^{\text{pred}} - \mathbf{u}_i^{\text{true}}, \quad \delta u_{i,\alpha} = u_{i,\alpha}^{\text{pred}} - u_{i,\alpha}^{\text{true}},$$

We use the following three complementary error measures:

$$\text{RMSE} = \left(\frac{1}{N} \sum_{i=1}^N \|\delta \mathbf{u}_i\|_2^2 \right)^{1/2} \quad \text{MAE} = \frac{1}{6N} \sum_{i=1}^N \sum_{\alpha=1}^6 |\delta u_{i,\alpha}| \quad \text{Rel. RMSE} = 100 \left(\frac{1}{N} \sum_{i=1}^N \frac{\|\delta \mathbf{u}_i\|_2^2}{\|\mathbf{u}_i^{\text{true}}\|_2^2 + \varepsilon} \right)^{1/2}$$

- **RMSE.** RMSE measures the Euclidean error of the rigid-body velocity vectors and therefore captures discrepancies in both magnitude and direction. Because errors are squared, velocity components with large errors dominate, and smaller components or slower particles may be obscured.
- **MAE.** We use componentwise MAE for training because each scalar velocity component contributes separately and errors are not squared. Larger components can still have greater influence, but much less so than with RMSE. Note that all 6 velocity components are trained together.
- **Rel. RMSE.** Relative RMSE normalizes each particle's vector error by the magnitude of its reference velocity before averaging, making slow and fast particles contribute more comparably. The normalization is applied to the velocity vector as a whole rather than to its individual components. Relative RMSE reports the normalized vector error as a percentage by multiplying the root-mean-square relative error by 100.

3. Experiments

We evaluate NeMO through a sequence of accuracy, validation, and performance tests. We begin with targeted accuracy studies that isolate the error of individual learned components, followed by tests of the fully assembled mobility operator in multi-particle configurations. We then compare the method against established validation cases from the Stokesian-dynamics literature before turning to runtime and scaling behavior.

All experiments are performed for suspensions of spherical particles in unbounded Stokes flow. In the initial configuration, particles do not overlap (however, in some dynamic simulations they are allowed to). Unless otherwise stated, model predictions are compared against reference rigid-body velocities computed with the MFS solver using 618 boundary nodes on each particle surface. Distances are non-dimensionalized by the particle radius a . For two particles at positions $\mathbf{x}_i$ and $\mathbf{x}_j$, their non-dimensional separation is defined as

$$L_{ij} = \frac{\|\mathbf{x}_i - \mathbf{x}_j\|}{a}. \quad (22)$$

Unless otherwise stated, we use normalized units in which the particle radius a and dynamic viscosity μ are both set to unity. Absolute velocity errors are reported in the resulting simulation units.

3.1. Accuracy

3.1.1. Individual Models

We start by looking at the accuracy of each neural network module on their respective test datasets, which consist of isolated particle pairs and local many-particle configurations. 20% of the generated data is reserved for testing, and the remaining 80% is used for training. For each sample, the force and torque directions are drawn independently and uniformly at random, while their magnitudes are fixed at $\|\mathbf{F}\|_2 = \|\mathbf{T}\|_2 = 6\pi$.

We first consider the two-body models. To simplify training, we train both two-body components on a shared dataset. Because $m_s^{(2)}$ and $m_t^{(2)}$ multiply the independent generalized forces $\mathcal{F}_i$ and $\mathcal{F}_j$, respectively, their contributions remain distinguishable during training. For each sample in this dataset, predicted target velocity is thus:

$$\mathbf{u}_i^{pred} = m^{(1)}(\mathbf{x}_i) \mathcal{F}_i + m_s^{(2)}(\mathbf{x}_i, \mathbf{x}_j) \cdot \mathcal{F}_i + m_t^{(2)}(\mathbf{x}_i, \mathbf{x}_j) \cdot \mathcal{F}_j \quad (23)$$

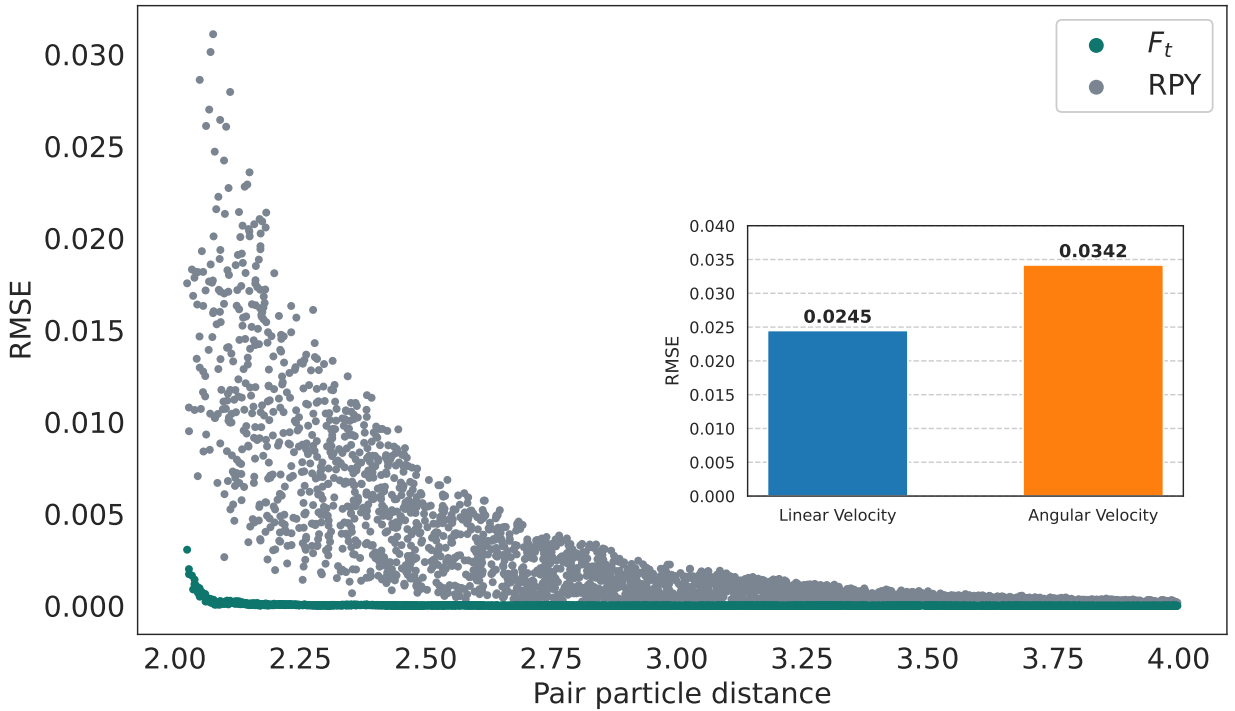


Figure 1: Accuracy of the learned two-body mobility models on the held-out pair test set. The main panel compares the sample-wise PRMSE of the learned cross-mobility model $m_t^{(2)}$ and the RPY approximation as a function of the dimensionless separation L_{ij} . Each point pools the error over the six rigid-body velocity components of one sample; multiple values may occur at the same separation because the source loading direction varies. The inset reports the test-set PRMSE of the full two-particle prediction in Eq. (23), separated into translational and rotational components.

This dataset uses a total of 30k samples. Both dataset generation and model training can be completed in an hour using mid-range CPUs and GPUs. The inset of Figure 1 shows the RMSE of the predicted two-body velocity contribution $\mathbf{u}_i^{pred}$, separated into translational and rotational components. The errors are well below 1% of the true velocity magnitude in both cases, indicating that the combined model accurately represents the two-particle mobility response.

The main panel compares the error of the learned cross-particle model $m_t^{(2)}$ with that of RPY on the test dataset. To isolate the contribution from $m_t^{(2)}$, the target-particle generalized force $\mathcal{F}_i$ is set to zero. For equal-radius spheres, the relevant tensor coefficients depend only on the inter-particle distance, but this dependence becomes highly nonlinear at short separations because of the rigid particle surfaces. This is precisely where the learned model provides the

largest improvement: near contact, RPY accumulates substantial error, while $m_t^{(2)}$ maintains a low RMSE. At larger separations, the two models converge toward similar accuracy as the interaction becomes increasingly dominated by smooth far-field behavior.

Next, we evaluate the many-body correction term, $m_t^{(n)}$. Recall that each training sample contains a source particle j , a target particle i , and K neighboring particles, where $1 \leq K \leq K_{\max}$ and all but source have zero force. The target velocity of each training sample is modeled as

$$\mathbf{u}_i^{\text{pred}} = m_t^{(2)}(\mathbf{x}_i, \mathbf{x}_j) \cdot \mathbf{F}_j + m_t^{(n)}(\mathbf{x}_i, \mathbf{x}_j, \mathcal{N}_{ij}) \cdot \mathbf{F}_j. \quad (24)$$

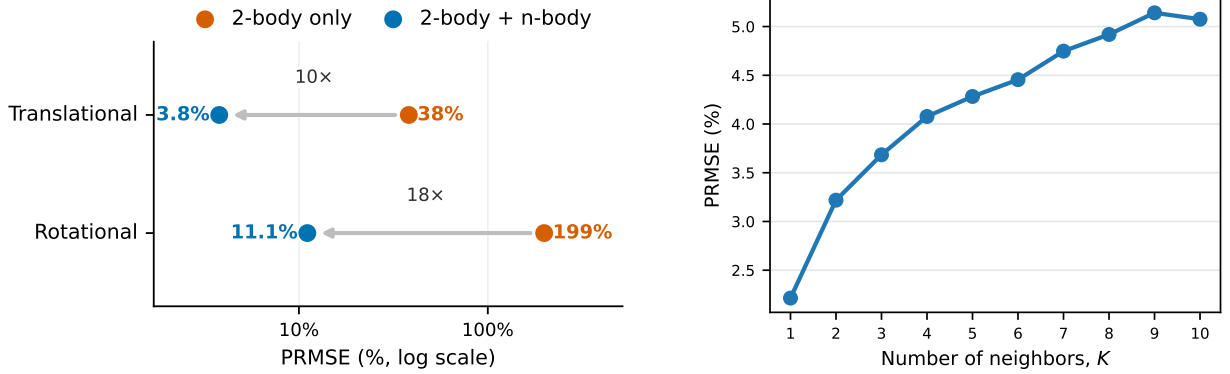


Figure 2: Accuracy of the many-body correction model for spherical particles. Left: Translational and rotational PRMSE for the two-body-only prediction and the combined two-body and many-body prediction. The arrows indicate the reduction in error obtained by including the many-body correction. Right: PRMSE of the combined model as a function of the number K of neighboring particles.

Figure 2 evaluates and analyzes the many-body correction model. Whereas the two-body model achieves sub-1% errors for the isolated pair configurations considered in Figure 1, its accuracy deteriorates substantially in the presence of passive neighboring particles. As shown in the left panel, the two-body-only prediction has a translational PRMSE of 38% and a rotational PRMSE of 199%. Including the many-body correction $m_t^{(n)}$ reduces these errors to 3.8% and 11.0%, respectively, corresponding to reductions of approximately 10 $\times$ for translational velocity and 18 $\times$ for rotational velocity. The right panel reports the PRMSE of the combined two-body and many-body model as a function of the number of neighboring particles $K = |\mathcal{N}_{ij}|$ present around the source–target pair. The error increases from 2.43% for $K = 1$ to 5.22% for $K = 10$. As K increases, the model must represent the coupled influence of a progressively more complex local hydrodynamic environment, making the collective correction more difficult to learn accurately. In all subsequent experiments, we fix the maximum neighborhood size to $K_{\max} = 10$, so that each source–target pair is represented using at most ten neighboring particles. A systematic study of how the accuracy and computational cost vary with $K_{\max}$ would be useful, but is left for future work.

3.1.2. Mobility Operator

We next evaluate the accuracy of the complete mobility operator M , assembled from all learned and analytical components, in randomly generated multi-particle configurations. Particle centers are sampled from a uniform spatial distribution. Accuracy is reported using a particle-normalized PRMSE. For each particle, we compute the RMSE across its six rigid-body velocity components, normalize it by the magnitude of the corresponding reference velocity, and then average the resulting values over all particles. This metric does not separately balance variations among the individual velocity components; instead, it prevents particles with larger velocity magnitudes from dominating the reported error. At each specified volume fraction, we report the median over ten independent runs. The volume fraction is defined as the total particle volume divided by the volume of the smallest bounding box enclosing the configuration.

Our first experiment measures how the accuracies of different mobility operators change with volume fraction. In this setup, the particles are subjected to unit forces and torques in random directions. We compare the full NeMO operator, denoted here as the NeMO n-body, against a variant that includes only self- and two-body interactions, i.e., NeMO 2-body which lacks the learned many-body correction term $m_t^{(n)}$. As a baseline, we include the RPY mobility, where pairwise two-body hydrodynamic interactions are summed directly using the RPY tensor in an $O(N^2)$ fashion, without any long-range acceleration. All operators use the same analytical self-interaction term.

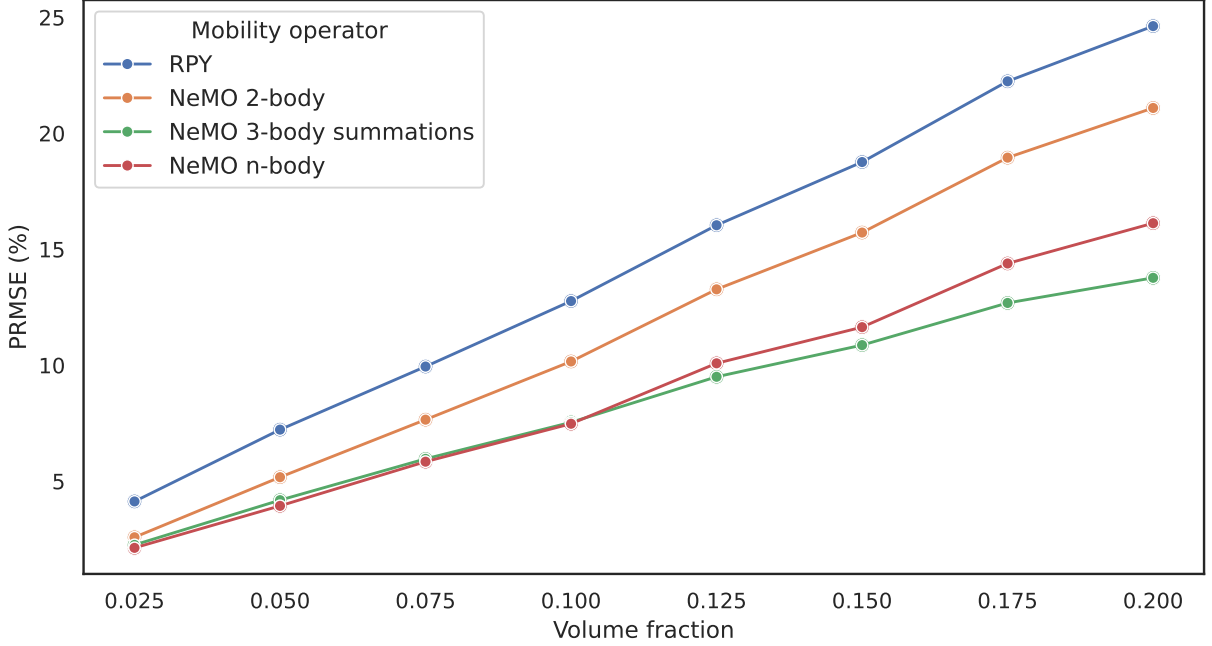


Figure 3: PRMSE of the assembled mobility operators as a function of particle volume fraction ϕ for uniformly distributed configurations with $N = 200$ particles. Each point reports the median over ten independently generated configurations with randomly oriented unit forces and torques

We also compare against a three-body correction strategy similar to [12], in which the many-body correction $m_t^{(n)}$ is replaced by a sum of learned triplet contributions. For this variant, we train a separate model that processes only one neighboring particle at a time, together with the source and target particles. Thus, for a source–target pair with K neighboring particles, the neighborhood interaction is decomposed into K source–target–neighbor triplets. The three-body network is evaluated once for each triplet, and the resulting velocity corrections are summed to obtain the total many-body contribution. We refer to this variant as NeMO 3-body; its self and two-body terms are kept identical to those of NeMO 2-body, isolating the effect of the many-body correction strategy. In both NeMO n-body and NeMO 3-body, neighborhood construction and preprocessing remain $O(K)$. The main difference lies in the learned evaluation: NeMO 3-body requires K network passes per source–target neighborhood, whereas NeMO n-body constructs a single neighborhood feature vector and evaluates it in one pass.

Figure 3 shows that the full mobility operator, NeMO n-body, significantly improves accuracy over the NeMO 2-body model, with the gap increasing as the volume fraction grows. The NeMO 3-body summation becomes slightly more accurate than NeMO n-body in the densest configurations. We attribute this crossover to the fixed neighborhood size $K_{\max} = 10$ used by the n-body model: at higher volume fractions, more than ten particles may lie within the cutoff region, so the model does not receive the complete local neighborhood, whereas the triplet-based model processes the neighboring particles individually. At 10% volume fraction, the full NeMO operator achieves a PRMSE of approximately 7.48%, compared with roughly 10.2% for the two-body-only model and 12.8% for RPY. Even without the many-body correction term, the learned two-body model remains consistently more accurate than RPY, despite

having comparable runtime, as will be shown in Section 3.3. This improvement is notable because all operators use the same analytical self-interaction term, which contributes the dominant part of the particle velocity. Interestingly, we found that without the short-range correction $m_s^{(2)}$, the improved accuracy of the two-body cross-interaction module $m_t^{(2)}$ does not translate as clearly to multi-particle configurations. This ablation is not shown in the figure. A similar observation was made in recent work by [14].

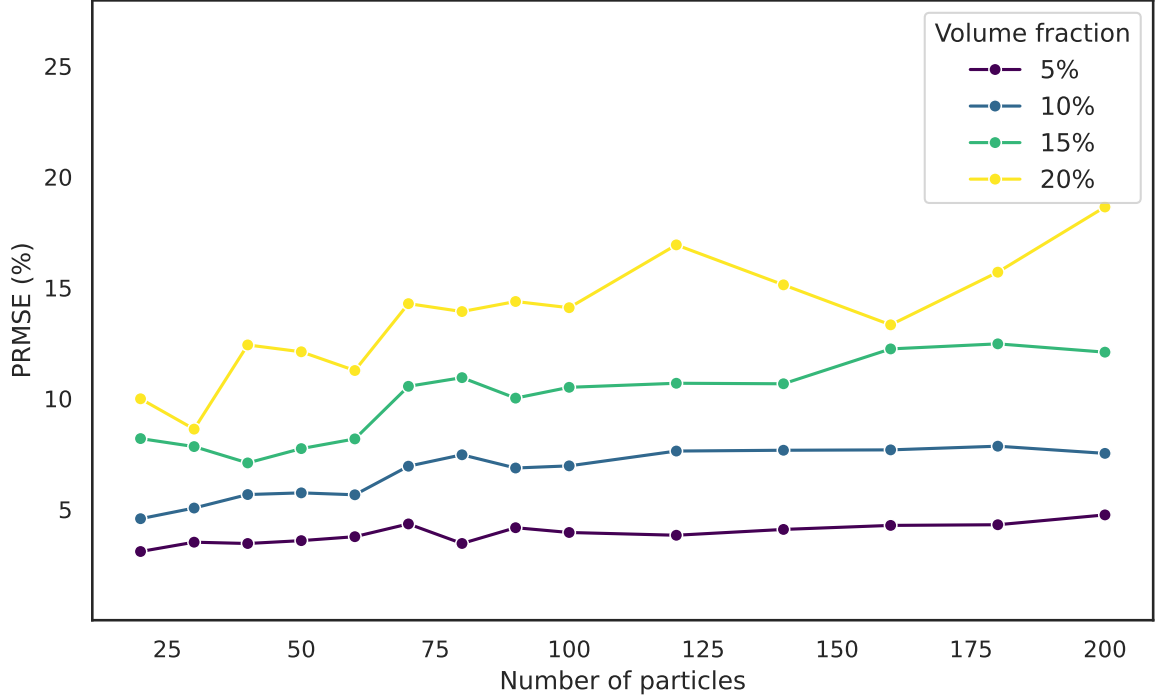


Figure 4: Grand mobility accuracy vs number of particles.

Next, we evaluate how the accuracy of the assembled mobility operator varies with the number of particles. Figure 4 reports the PRMSE for uniformly distributed particle configurations in several volume fractions. For a fixed volume fraction, the error remains roughly flat as the number of particles increases, especially at lower volume fractions. At higher volume fractions, the curves show larger fluctuations and a mild upward trend. The dominant trend, however, is the increase in error with volume fraction, rather than with particle count.

We have so far looked at an error metric that condenses the error over all particles and their components into a single value. This can hide interesting information. To get a more detailed picture of the error distribution, and also to expose certain systematic biases of NeMO, we design a simple experiment: 300 particles are placed around origin at 10% volume fraction. Particles at $X < 0$ get unit force in positive y direction, and particles $X > 0$ get unit force in negative y direction, and resulting velocities are computed. We repeat this experiment 10 times and use Kernel Density Estimation (KDE) to obtain a smooth estimate of the error distribution. Unlike random directional force, this experiment creates strong collective motion and long-range hydrodynamic interactions become more important.

Figure 5 provides a spatial view of the predicted velocity field and the corresponding error distribution. The top-left panel shows the KDE-smoothed velocity field generated by the imposed forcing, with the background color indicating the local velocity magnitude. The lower-left panel reports the absolute RMSE and shows a clear correlation with the velocity magnitude: regions of stronger motion also exhibit larger absolute errors. The lower-right panel shows the corresponding error normalized by the local MFS velocity magnitude. In this normalized view, the trend reverses, with the largest relative errors appearing in the weak-flow region. This behavior is also consistent with the training procedure: because all velocity components are optimized through a shared loss, the larger velocity signals tend to dominate, while smaller-magnitude components are learned less accurately.

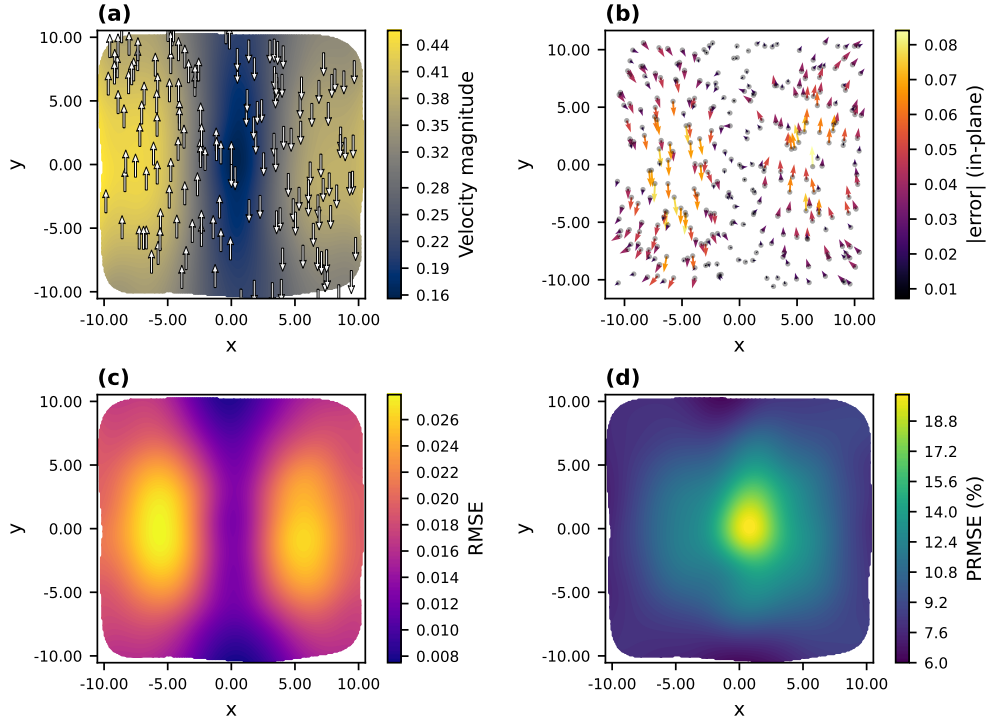


Figure 5: Spatial structure of the NeMO prediction and its error in the opposing-force experiment. (a) KDE-smoothed field of the predicted translational-velocity magnitude, with arrows indicating the applied-force directions. (b) In-plane error vectors, $\mathbf{e}_i = \mathbf{u}_i^{\text{MFS}} - \mathbf{u}_i^{\text{NeMO}}$, for a representative configuration; arrow direction and length encode the error vector, and color indicates $\|\mathbf{e}_{i,xy}\|_2$. (c) KDE-smoothed field of the translational-velocity RMSE. (d) Corresponding relative RMSE, normalized by the magnitude of the MFS reference velocity. The KDE fields are estimated from ten independently generated configurations.

The top-right panel shows the error vectors, $\mathbf{e}_i = \mathbf{u}_i^{\text{MFS}} - \mathbf{u}_i^{\text{NeMO}}$, for one representative configuration. Comparing this panel with the velocity field in the top-left panel, the error vectors are typically directed opposite to the local velocity, indicating that NeMO slightly overpredicts the velocity magnitude. We attribute this bias to the finite cutoff used for the many-body correction. In classical Stokesian Dynamics, nearby particles partially screen long-range hydrodynamic interactions, thereby reducing the net velocity response transmitted over longer distances [5]. In NeMO, this screening effect is learned only from neighbors within the cutoff radius, while particles outside the cutoff are treated using a pairwise far-field approximation. Consequently, part of the collective slowdown is not captured, appearing here as a systematic overprediction of the velocity magnitude.

3.2. Validation Tests

Next, we put our grand mobility operator to the test in a few classical Stokesian dynamics setups.

3.2.1. Horizontal Line-of-Spheres:

We begin with the horizontal line-of-spheres test case [5]. Here, 15 spheres are arranged on a line with uniform center-to-center spacing of 4.0. A unit force perpendicular to the line is applied to every sphere, and we measure the instantaneous drag coefficient, $\lambda = F/(6\pi\mu aU)$, shown in Figure 6. We index the displayed half-chain from the outermost sphere (0) to the central sphere (7). This configuration is a useful test of long-range hydrodynamic coupling along the chain: every sphere sediments faster than it would in isolation ($\lambda < 1$), and the enhancement is strongest for the central spheres, which are assisted by neighbors on both sides. The drag coefficient is therefore largest at the ends of the chain and decreases steadily toward the center.

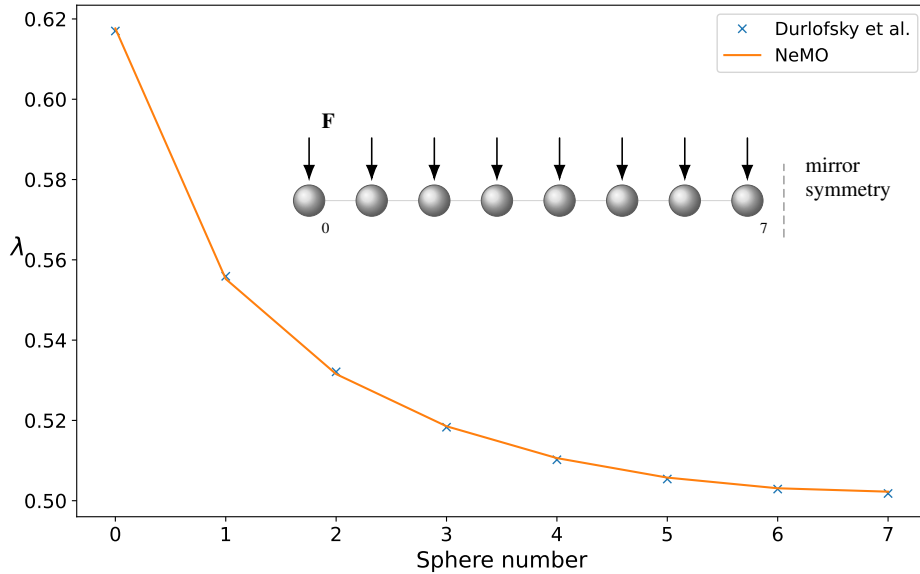


Figure 6: Drag coefficient for 15 spheres on a line with center-to-center distance 4.0. Only half of the symmetric configuration is shown, indexed from the outermost sphere (0) to the central sphere (7). The reference data are from Durlofsky et al. [5].

We only show half the spheres because the configuration is symmetric about the center. The predicted curve is nearly indistinguishable from the reference data, including the sharp rise at the chain ends.

3.2.2. Spheres on Equilateral Triangle

This test case is adapted from [37]. Here, 3 spheres are placed on an equilateral triangle of varying lengths. A unit force equal to $F = 6\pi\mu a$ is applied to one of the particles (lets call it p_1) towards the centroid of the triangle. The two other particles are force-free and torque-free. In an isolated setting, this would lead to unit velocity. This test case is

interesting because of the strong lubrication effect when particles are close. The particle being forced will experience a strong resisting force from the other two particles, leading to a much reduced velocity. There is also strong two body and 3-body effects.

We again measure the instantaneous velocity of all three particles and compare it with the data reported by [37]. The paper also notes the output of Stokesian Dynamics (SD) baseline with lubrication correction, we include that as well for comparison. In Figure 7, U_1 denotes the velocity of particle p_1 along the direction of force, U_2 and U_3 denote the velocity of the other two particles along and transverse to the force direction respectively (magnitude is equal for both particles). Ω denotes the induced angular velocity on both particles.

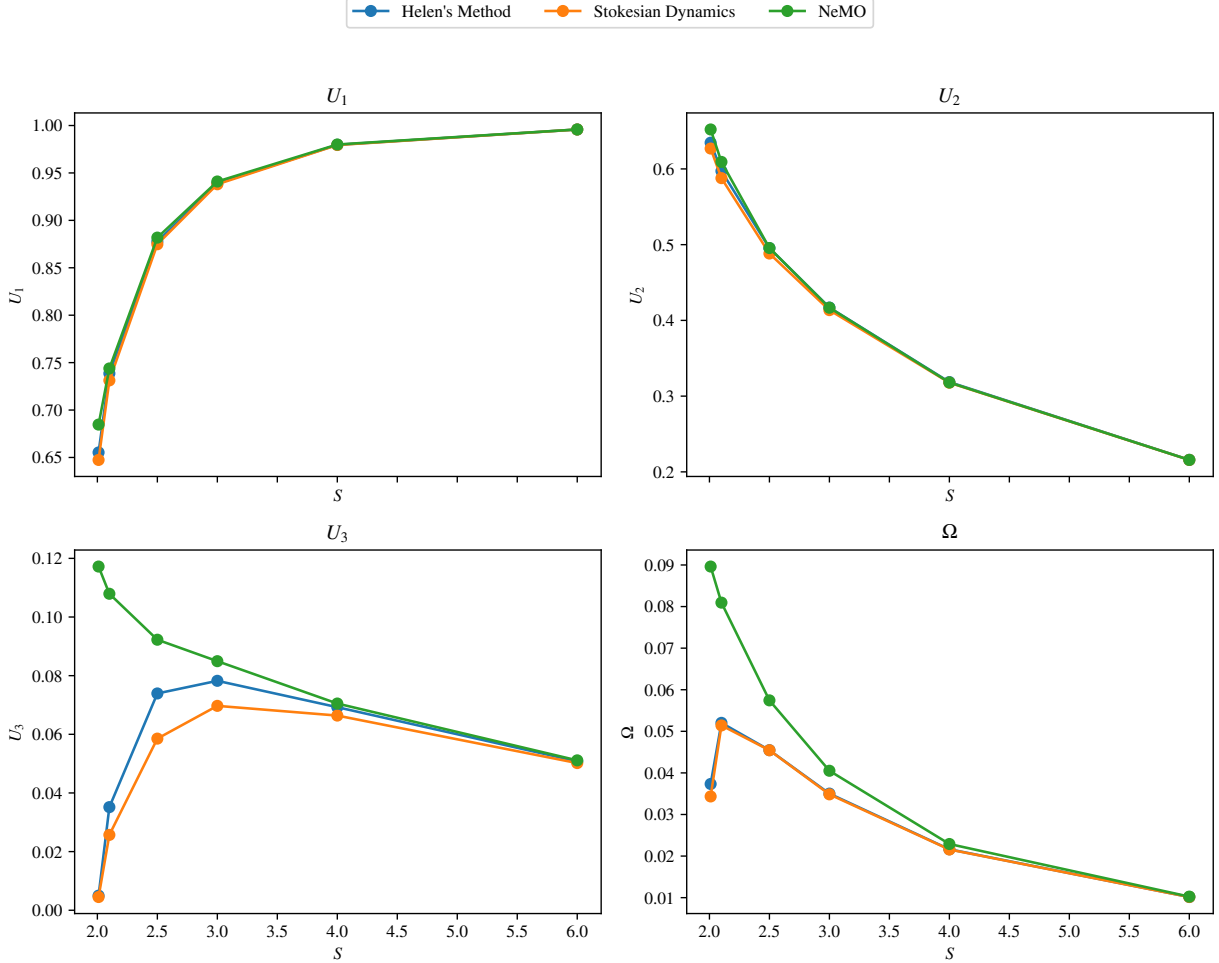


Figure 7: Mobility operator on the 3-sphere equilateral triangle testcase. The reference is from Wilson et al. [37]. Stokesian Dynamics method is from [24].

This testcase exposes both the strengths and the limitations of NeMO. It predicts resisting motion of particle p_1 (U_1) across the entire distance range quite well, with a noticeable deviation only at the tightest gap ($L = 2.01$, i.e., a 1% surface gap). This validates the inclusion of the $m_s^{(2)}$ component, as a simple pairwise operator (such as RPY) without correction to the diagonal block would predict $U_1 \approx 1$ at all separations and therefore miss the lubrication slowdown entirely. The induced velocity on particle p_2 and p_3 along the forcing direction (U_2) is likewise captured quite accurately, including the near-contact regime.

The weak spots are the transverse response U_3 and the rotational velocity Ω . These quantities are harder to capture in the near-contact regime, where the hydrodynamic interactions between nearly touching spheres become

more complicated. In addition, both signals are relatively small in magnitude, so a typical loss function is dominated by U_1 and U_2 and therefore naturally places less emphasis on these smaller terms during training.

3.2.3. Dynamic Simulation: Falling Particle Cloud

We next examine how errors from the approximate mobility operator affect a long-time dynamic simulation. As a qualitative benchmark, we consider the falling particle cloud experiment of Metzger et al. [38]. The particles are initialized randomly inside a spherical region, forming an approximately uniform suspension cloud that settles under gravity in a viscous fluid.

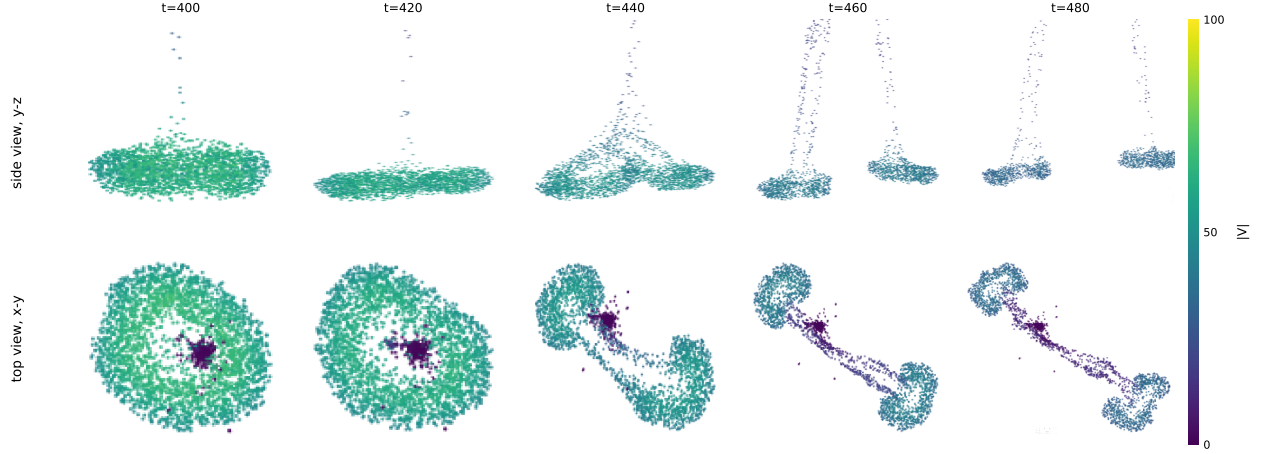


Figure 8: Dynamic sedimentation of a cloud of $N_0 = 3000$ particles under gravity, following the experimental configuration of Metzger et al. [38]. Side and top projections show the characteristic destabilization sequence: particle leakage from the rear of the cloud, flattening into an oblate shape, torus formation, and eventual breakup into secondary clusters. Particle color denotes the velocity magnitude $|\mathbf{V}|$.

Figure 8 shows that NeMO reproduces the main qualitative features of the falling-cloud instability. The initially compact cloud first develops a trailing leakage region as particles escape from the rear. As settling continues, the cloud flattens, rolls into a toroidal structure, and eventually separates into secondary clusters. These daughter clusters then undergo similar deformation, producing the familiar cascade observed in falling suspension clouds.

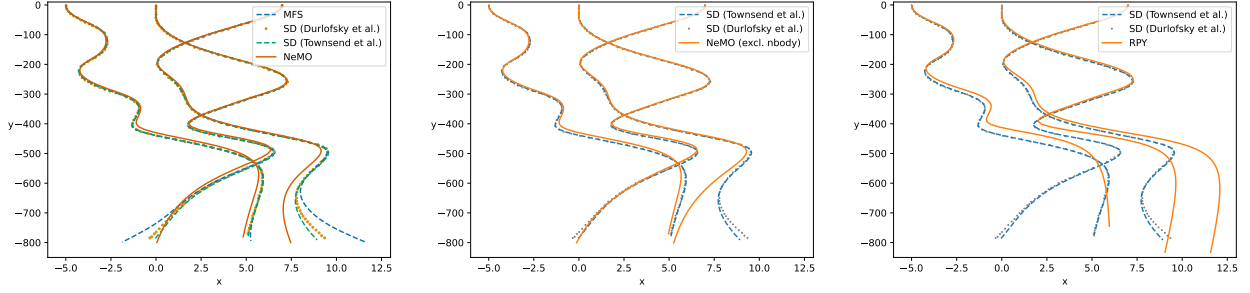
This test is best interpreted as a long-time, qualitative validation of the far-field dynamics rather than a stringent test of the full NeMO architecture. The particle cloud remains relatively dilute, and the macroscopic instability is largely governed by long-range hydrodynamic interactions; indeed, even point-particle models can reproduce this sequence at a qualitative level. With a MAC parameter of 0.3, the far-field component preserves the large-scale hydrodynamic structure needed to reproduce the observed instability pathway.

3.2.4. Dynamic Simulation: 3 Falling Spheres

We now consider a dynamic simulation of 3 unit spheres falling under gravity, a test adapted from [5]. The spheres are placed on the x -axis, and a unit gravitational force is applied to each sphere in the negative z direction. We simulate the system for 13000 timesteps using a 4th-order Runge-Kutta integrator with timestep size $\Delta t = 1$. This complements the previous test as it focuses solely on the near-field components.

Long simulations like this are sensitive to initial conditions and small perturbations. We include 3 reference simulations here to highlight the uncertainty region and compare the trajectories with NeMO. The references are: Stokesian Dynamics from [5], another implementation of SD from [24], and method of fundamental solutions [3].

Figure 9 compares the trajectories for the 3 spheres (in terms of their x and y coordinates over time) obtained with the full learned mobility operator M_{nb} (left), the learned two-body operator M_{2b} (middle), and RPY (right). In the left-most panel, we can see that NeMO and reference methods align remarkably well for $y > -400$. After that, the trajectory of the rightmost sphere predicted by NeMO starts to diverge, and after $y < -700$ the difference becomes more pronounced, although this is also the region where the reference solutions begin to diverge from one another.



(a) Using the full learned mobility operator M_{nbody} . (b) Using learned two-body model as mobility operator. (c) Using RPY as mobility operator.

Figure 9: Comparison of different mobility operators on the dynamic 3-sphere falling simulation. Left: full learned mobility operator M_{nbody} . Middle: learned two-body model. Right: RPY.

All three operators still capture the overall swirling pattern of the falling spheres. The middle panel shows that M_{2b} preserves this qualitative behavior but accumulates larger error, mainly in the trajectory of the rightmost sphere. By contrast, the right panel shows that a pairwise operator with the RPY kernel breaks down earlier and fails to reproduce the overall pattern once the simulation moves past $y < -400$.

3.3. Performance

We now turn from accuracy to performance. As discussed in the introduction, a key objective in NeMO’s design is to capture richer near-field physics while keeping the runtime comparable to simple pairwise operators on modern parallel hardware, especially GPUs. To this end, NeMO is designed to expose a large amount of fine-grained parallelism while keeping the dominant kernel’s arithmetic-intensive enough to make effective use of GPU compute resources. The learned near-field components replace some low-cost analytical pair interactions with denser tensor-core-friendly computations. At the same time, the operator is assembled from many independent pair and local-neighborhood evaluations, allowing the workload to be distributed across a large number of GPU threads.

Unless specified otherwise, all benchmarks were run in PyTorch/CUDA on an NVIDIA H200 GPU for spherical suspensions at 10% volume fraction. We use `torch.compile` with autotuning set to `max` to encourage kernel fusion and high throughput. The RPY baseline uses FP32 throughout, while the neural network modules use TF32 tensor-core matrix multiplies with FP32 accumulation. Velocity accumulation is performed in double precision. These are the same numerical precisions used in the previous subsection for the accuracy results.

3.3.1. Two-Body: RPY vs Neural Net

We begin with the two-body cross-interaction kernel, our learned replacement for the RPY off-diagonal mobility block. This isolated benchmark is useful because it separates the cost of the learned pair model from the rest of the mobility-operator pipeline. This is also the simplest setting in which the performance argument behind NeMO becomes visible.

Figure 10 shows that the neural network is slower than RPY at small batch sizes, but the gap narrows as the workload grows. At the largest batch size, RPY reaches 1023 million pair evaluations per second, while the neural network reaches 495 million pair evaluations per second, corresponding to roughly a $2\times$ slowdown.

This gap is small relative to the difference in arithmetic complexity: the RPY kernel requires only 96 floating-point operations per pair, whereas the neural network uses about 11,500. The much smaller runtime gap reflects the hardware regime we target. TF32 tensor cores, reuse of weights and activations in fast on-chip memory, and kernel fusion make the additional neural-network computation much cheaper than the raw FLOP count alone would suggest.

This isolated pair-kernel benchmark should therefore be interpreted as a worst-case view of the learned model’s cost. In a practical mobility evaluation, the pair kernel is only one part of the pipeline; neighbor search, graph construction, data movement, and long-range interactions also contribute to the total runtime. As shown in the next subsection, once these full-operator costs are included, the $2\times$ kernel-level gap translates to only a small single-digit-percent difference in end-to-end runtime.

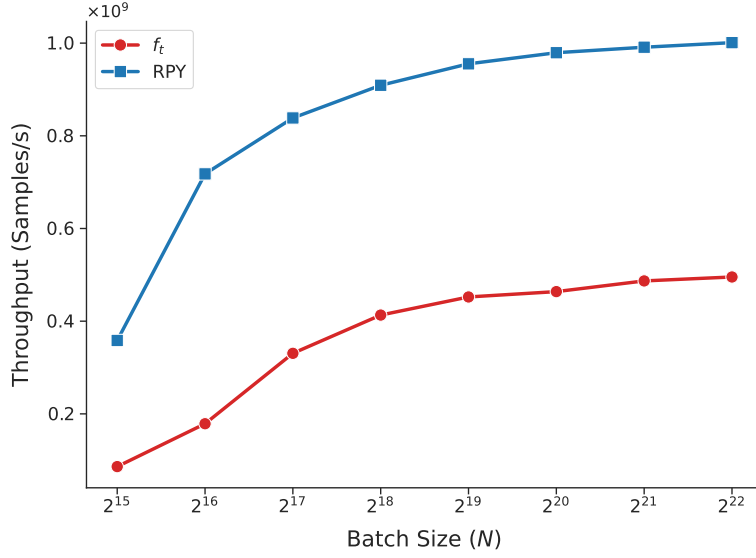


Figure 10: Throughput comparison between the analytical RPY kernel and the learned two-body neural network for evaluating pairwise cross interactions.

3.3.2. Performance Scaling

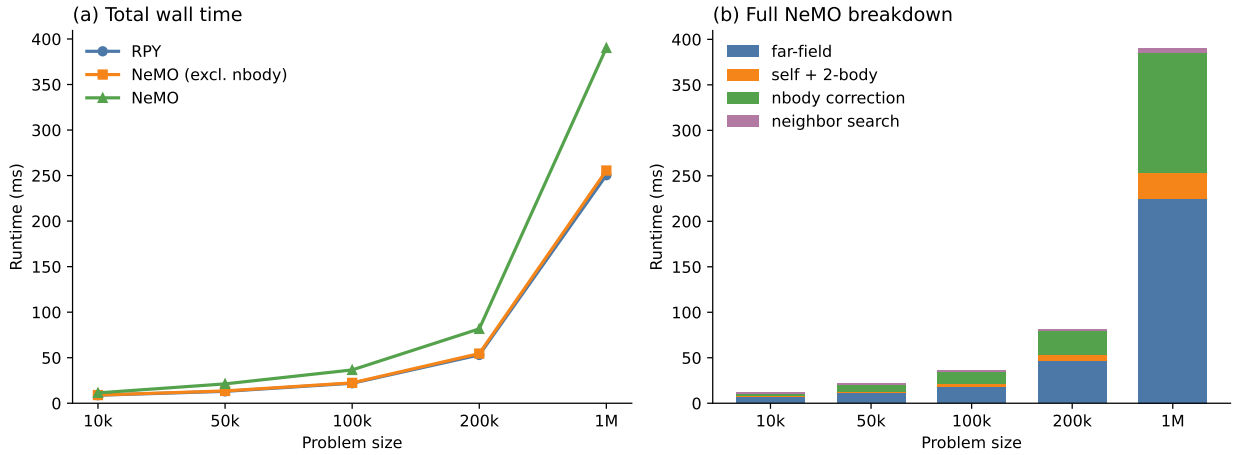


Figure 11: Runtime breakdown of mobility application on an H200 GPU at 10% volume fraction. For each problem size, we compare the RPY baseline, the two-body NeMO variant, and the full NeMO operator with the many-body correction.

Next, we take a closer look at how the cost of the full mobility operator is distributed and how that balance changes with problem size. We benchmark the operator on uniformly distributed suspensions at 10% volume fraction. In this regime, the neighbor list contains about 22 neighboring particles per particle on average within the r_c cutoff. For each problem size, we first run a few warmup evaluations so that `torch.compile` can specialize the kernels, and then report the average of five timed runs for one mobility application.

Figure 11 shows how different operators and different components of NeMO scale. On the left panel, we see that indeed once fixed runtime costs such as neighbor search and treecode acceleration is factored in, NeMO with both 2-body components ($m_t^{(2)}$ and $m_s^{(2)}$) has single-digit percentage slowdown compared to just RPY. The addition of the

n-body term adds around 50% overhead in large scale simulations ($>100k$).

The right panel breaks the runtimes down into components. Note that the total near-field runtime is controlled mainly by local density: for a fixed cutoff, the number of nearby interactions grows approximately in proportion to the volume fraction ϕ , not the global system size. By contrast, the far-field solver depends on the full suspension and therefore grows with N , with the slight superlinear trend expected from the $O(N \log N)$ complexity of our LB-VH/treecode traversal. Consequently, at fixed ϕ , a larger fraction of the total runtime is spent in M_{ff} as N increases. Within M_{nf} , the dominant cost is clearly the many-body correction. This matches the algorithmic structure: the self term is $O(N)$, the two pairwise near-field terms are $O(NK)$, and the many-body correction is $O(NK^2)$, where K is the local neighbor count. Neighbor search remains a relatively small part of the total runtime throughout.

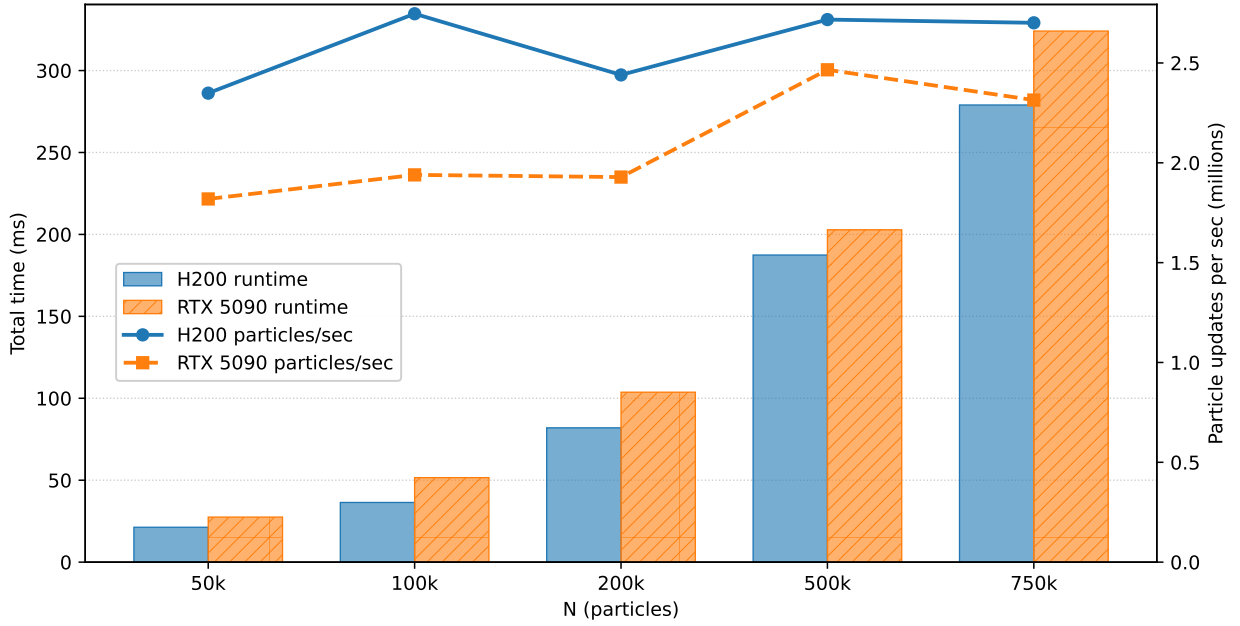


Figure 12: End-to-end scaling of the grand mobility operator on NVIDIA H200 and RTX 5090 GPUs. Bars show total runtime for one mobility application, and lines show the corresponding particle-update rate.

Figure 12 shows the end-to-end scaling up to 7.5×10^5 particles. The total runtime grows as expected- in the fixed-density regime considered here, however, K stays roughly constant, so the observed scaling is close to linear in N . This is reflected in the throughput curves: on the H200, the code sustains roughly 2.4 to 2.7 million particle updates per second across the full range. In other words, in a suspension with about 2.7×10^6 particles, one application of NeMO operator takes about a second.

The RTX 5090 follows the same overall trend and remains surprisingly close to the H200, which is encouraging because consumer GPUs are much cheaper and more widely available. This is despite the H200's much stronger hardware specs: its memory bandwidth is 4.8 TB/s, compared with 1.8 TB/s for the RTX 5090, a $2.7\times$ gap, and its FP64 throughput is about $21\times$ higher. By contrast, the RTX 5090 is much closer to the H200 in single- and reduced-precision compute, with higher nominal FP32 throughput (~ 105 versus 67 TFLOP/s), but lower dense TF32 Tensor Core throughput (~ 105 versus ~ 495 TFLOP/s). These differences suggest that the measured runtime is governed less by memory bandwidth or FP64 arithmetic alone and more by a mix of FP32/TF32 computation and latency-sensitive stages such as tree construction, prefix sums, and neighbor-list assembly. It is worth noting, however, that the current implementation executes the NeMO modules sequentially and does not use the larger H200 memory capacity for module-level task parallelism; exploiting this additional headroom would likely increase the gap between the H200 and the RTX 5090.

3.4. Sedimentation Benchmark

Lastly, we consider the large-scale sedimentation benchmark introduced in H-HIGNN [21]. The problem consists of two suspension drops sedimenting under gravity in an unbounded domain. Each drop is initially spherical with radius 175 and particle volume fraction 10%, and contains 523,305 particles, for a total of 1,046,610 particles. A uniform force $F = [0, 0, -1]$ is applied to every particle, and the system is advanced with an explicit Euler integrator using $\Delta t = 0.01$. This is a useful stress test because the particle distribution quickly becomes highly non-uniform as the trailing drop elongates, catches up with the leading drop, and the two clouds begin to coalesce.

Figure 13 shows that we successfully reproduce the qualitative behavior reported in [21]: the trailing drop stretches into a narrow column, penetrates the leading drop, and then forms the same coalescing structure seen in the reference simulation. This benchmark is therefore not only a large- N performance test, but also a check that the learned mobility operator remains stable in a strongly inhomogeneous sedimentation flow.

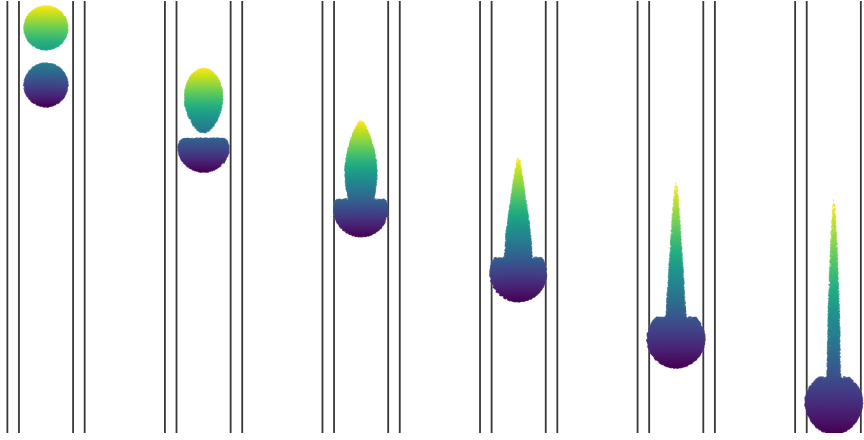


Figure 13: Reproduction of the large-scale two-drop sedimentation benchmark from H-HIGNN [21]. Two initially separated suspension drops, containing more than one million particles in total, sediment and coalesce over time. From left to right, the snapshots show the particle distribution at $t = 0.0, 0.2, 0.4, 0.6, 0.8$, and 1.0 .

On an H200 GPU, each time step takes 0.48 seconds. The far-field treecode component M_{ff} accounts for 283 ms ($\approx 61\%$ of the step), while the near-field stage M_{nf} costs 182 ms ($\approx 39\%$). The first 150 steps of the simulation, as shown in Fig. 13, completes in 74 seconds. Note that as the simulation progresses, the particle distribution becomes more non-uniform, which leads to a slight increase in the far-field runtime due to the treecode traversal. The near-field runtime remains relatively stable throughout, as it depends mainly on local density rather than global distribution.

The performance advantage NeMO demonstrates here over the existing baseline is substantial. For a like-for-like comparison with H-HIGNN [21], we run this benchmark on an NVIDIA RTX A4500 GPU (~ 20 GB VRAM). Our implementation fits comfortably on a single GPU, whereas H-HIGNN requires four RTX A4500 GPUs for the same problem. H-HIGNN reports 28.8 s per time step on four GPUs, corresponding to roughly 24 min for 50 steps [21]. By comparison, our implementation requires only 2.6 s per step on a single RTX A4500, or approximately 132 s (≈ 2.2 min) for 50 steps. Thus, NeMO delivers roughly an $11\times$ wall-clock speedup while reducing hardware usage by a factor of four. The observed speedup may reflect advantages across multiple parts of the design. On the long-range side, NeMO is designed to interface naturally with existing fast solvers such as treecodes, so the global hydrodynamic contribution can be evaluated without placing the full problem inside a distributed neural-network pipeline. This improves memory efficiency, keeps the simulation within a single-GPU setting, and avoids the inter-GPU communication overhead associated with the distributed H-HIGNN setup. On the near-field side, NeMO represents the many-body correction through a single neighborhood-based neural-network pass. In contrast, the additive three-body formulations of H-HIGNN require $O(K)$ neural-network evaluations per pair-particle neighborhood, where K denotes the number of relevant local triplets.

Another GPU-focused method worth placing on the broader fidelity–cost hierarchy is the Fast Stokesian Dynamics (FSD) method of Fiore et al. [18], along with its modern JAX-based implementation, JFSD [39]. This class of

methods lies in a higher-fidelity, higher-cost regime than NeMO, since it retains a Stokesian Dynamics formulation with analytical lubrication corrections and explicit stresslet resolution. It is therefore not directly comparable to our learned mobility approximation, both because the hydrodynamic model is different and because the reported particle-throughput numbers correspond to periodic simulations in which the long-range interactions are accelerated using a spectral Ewald method. Nevertheless, these results provide useful context for the cost of higher-fidelity GPU-based stokesian dynamic methods. Fiore et al. reported reaching 6×10^4 particle updates per second on a GTX 980 Ti using a hand-tuned CUDA implementation. More recently, JFSD reported throughput of up to roughly 2×10^4 particle updates per second on an RTX 4060 using JAX. On the same RTX 4060 GPU, our PyTorch-based sedimentation code reaches approximately 4.4×10^5 particle updates per second.

3.5. Discussion

Taken together, these experiments support the key value proposition of NeMO: richer physical dynamics at a computational cost comparable to that of the simplest pairwise mobility operator on GPUs.

Evaluating this performance requires a diverse, complementary suite of tests, as no single test can truly capture the overall capability of a mobility operator. We started with individual components, then moved to NeMO’s overall accuracy at different volume fractions and system sizes, demonstrating its ability to keep the error within a single-digit percentage relative to the true velocity. However, while aggregate metrics such as RMSE or PRMSE over all particles provide a quick, high-level snapshot, they can obscure critical near-field dynamics. Phenomena such as lubrication and three-body interactions typically contribute only small magnitudes to the overall velocity, making them easy to miss in global error calculations. To obtain deeper insight, we visualized how the error correlates with local velocity and density, as shown in Figure 5. Additionally, the three-sphere test of Wilson et al. [37] helped us zoom in on NeMO’s strengths and weaknesses. In the dynamic simulations, the falling-particle cloud primarily stressed the long-range component, while the falling-spheres test provided a complementary near-field validation, demonstrating how more accurate treatment of near-field dynamics can improve simulations over long time horizons.

The performance tests show that, across a wide range of simulation sizes, these accuracy benefits come at a modest computational cost premium. We believe this is not the product of any single performance trick. In fact, NeMO is written entirely in high-level Python-based frameworks. The modular architecture of NeMO allows us to make careful, cost-appropriate design decisions for each component within the framework. This is why parts of our framework forego learned components and stick to traditional analytical ones (self-interaction and long-range). This modularity also allows easy extensibility, another crucial contributor to performance. For example, when we struggled to obtain good performance from existing long-range solvers on GPUs, it was easier to swap out FMM for a bespoke GPU-resident treecode better matched to NeMO’s moderate-accuracy regime and highly non-uniform particle distributions.

Several additional optimization opportunities remain. Some isolated hotspots could be rewritten in low-level CUDA; the feature-engineering code for the n-body term is a good candidate. Another potentially significant optimization is task-based parallelism. In the current implementation, all near-field and far-field components are executed sequentially, so the total runtime is effectively their sum. But once their inputs are assembled, these components are naturally independent, and on larger GPUs they could in principle be executed concurrently, yielding a wall time closer to $\max(T_{\text{near}}, T_{\text{far}})$ rather than $T_{\text{near}} + T_{\text{far}}$. Finally, although we have not considered distributed-memory execution in this work, such an extension would not require developing the long-range solver from scratch. NeMO could instead build on the substantial existing literature on distributed fast multipole methods and parallel treecodes, including kernel-independent and Stokes-specific variants [40, 41].

4. Concluding Remarks and Future Work

We have presented NeMO, a learned mobility operator for Stokesian dynamics that combines structured near-field modeling with a scalable treatment of long-range hydrodynamic interactions. The formulation decomposes the mobility into self, pairwise, and many-body contributions, preserves reciprocal symmetry by construction, and is designed to interface naturally with fast far-field solvers. NeMO significantly improves physical fidelity over much simpler operators while adding only a modest computational cost.

The numerical results indicate that the proposed operator attains a level of accuracy sufficient for practical use in a broad class of problems. Across a range of validation cases, the method reproduces reference mobility behavior with

generally small errors, captures lubrication-dominated and collective effects that are not well represented by simpler pairwise approximations, and remains effective in full many-particle sedimentation simulations. The performance results further show that these gains can be achieved at a computational cost comparable to that of RPY-based pairwise approaches, enabling simulations at the scale of millions of particles on a single device.

Despite these capabilities, the current formulation entails specific methodological trade-offs. Most notably, while the operator is symmetric by construction, positive definiteness is not enforced analytically. Enforcing global positive-definiteness constraints while constructing the mobility operator piecemeal from localized geometries remains a significant mathematical challenge. Additionally, NeMO relies on a relatively crude lubrication approximation ($m_s^{(2)}$). This is a deliberate design choice: achieving higher lubrication accuracy, as conventional SD approaches do, typically requires inverting the mobility matrix, either explicitly or through iterative algorithms, which the present framework explicitly seeks to avoid. It is also worth noting that even these approaches do not consistently prevent rigid-surface overlap and may require explicit linear-complementarity-based methods. Such methods introduce an additional outer iteration and further increase computational cost [4, 25]. On the computational side, the throughput-oriented design of NeMO has two implications. First, its performance is likely to be less appealing on CPUs, where modest core counts cannot fully exploit the available fine-grained parallelism. Second, even on GPUs, NeMO is unlikely to provide a substantial improvement for small systems. A useful next step would therefore be a direct benchmark between NeMO and high-fidelity Stokesian Dynamics solvers on identical problem setups. Such a comparison would help clarify where each method lies on the cost–accuracy spectrum and which solver is most appropriate for a given simulation need.

Beyond these limitations, several natural extensions could be incorporated into future versions of NeMO while preserving its core structure. One is the treatment of polydisperse suspensions, for which the learned representation would need to accommodate variations in particle radius within a single model. Another is the extension to broader classes of particle geometries. The tensorial structure used here is compatible with axisymmetric particles in principle, and recent work on neural surrogates for spheroidal and helicoidal particles suggests that learned hydrodynamic models can also be effective for more complex shapes [42]. Achieving comparable accuracy within the present framework, however, would likely require richer geometric features, deeper networks, and larger training datasets. It would also be useful to extend NeMO beyond unbounded Stokes flow. Many applications involve periodic domains, walls, or other confining boundaries, where the relevant Green’s functions change, along with both the near- and far-field components of the mobility operator.

Even without these extensions, the present results suggest that NeMO is nearly as fast as simpler pairwise, non-iterative mobility approximations, substantially faster than similarly accurate ML-based baselines, and orders of magnitude faster than more accurate SD approaches. For the class of simulations considered here, NeMO occupies a significant region of the cost–accuracy spectrum in which it achieves state-of-the-art performance.

References

- [1] E. Corona, S. Veerapaneni, Boundary integral equation analysis for suspension of spheres in stokes flow, *Journal of Computational Physics* 362 (2018) 327–345.
- [2] A. Broms, A. H. Barnett, A.-K. Tornberg, A method of fundamental solutions for large-scale 3d elastance and mobility problems: A. broms et al., *Advances in Computational Mathematics* 51 (5) (2025) 45.
- [3] J. J. Jordan, D. A. Lockerby, The method of fundamental solutions for multi-particle stokes flows: Application to a ring-like array of spheres, *Journal of Computational Physics* 520 (2025) 113487.
- [4] W. Yan, E. Corona, D. Malhotra, S. Veerapaneni, M. Shelley, A scalable computational platform for particulate stokes suspensions, *Journal of Computational Physics* 416 (2020) 109524.
- [5] L. Durlafsky, J. F. Brady, G. Bossis, Dynamic simulation of hydrodynamically interacting particles, *Journal of fluid mechanics* 180 (1987) 21–49.
- [6] J. Rotne, S. Prager, Variational treatment of hydrodynamic interaction, *The Journal of Chemical Physics* 50 (11) (1969) 4831–4837.
- [7] H. Yamakawa, Transport properties of polymer chains in dilute solution: Hydrodynamic interaction, *The Journal of Chemical Physics* 53 (1) (1970) 436–443.
- [8] E. Wajnryb, K. A. Mizerski, P. J. Zuk, P. Szymczak, Generalization of the rotne–prager–yamakawa mobility and shear disturbance tensors, *Journal of Fluid Mechanics* 731 (2013) R3.
- [9] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, A. Anandkumar, Neural operator: Learning maps between function spaces with applications to pdes, *Journal of Machine Learning Research* 24 (89) (2023) 1–97.
- [10] A. Hashemi, A. Izadkhah, A graph neural network simulation of dispersed systems, *Machine Learning: Science and Technology* 6 (1) (2025) 015044.

- [11] L. Jbara, A. Goyal, A. Wachs, Graph neural network based model of hydrodynamic closure laws in non-spherical particle-laden flows, *Computers & Fluids* (2025) 106876.
- 835 [12] Z. Ma, Z. Ye, W. Pan, Fast simulation of particulate suspensions enabled by graph neural network, *Computer Methods in Applied Mechanics and Engineering* 400 (2022) 115496.
- [13] D. Hankins, J. Moskowicz, F. Stillinger, Water molecule interactions, *The Journal of Chemical Physics* 53 (12) (1970) 4544–4554.
- [14] K. W. Torre, J. de Graaf, Hydrodynamic interactions in particle suspensions: A perspective on stokesian dynamics, *arXiv preprint arXiv:2503.16083* (2025).
- 840 [15] F. Fuchs, D. Worrall, V. Fischer, M. Welling, Se (3)-transformers: 3d roto-translation equivariant attention networks, *Advances in neural information processing systems* 33 (2020) 1970–1981.
- [16] R. Yokota, L. Barba, Comparing the treecode with fmm on gpus for vortex particle simulations of a leapfrogging vortex ring, *Computers & Fluids* 45 (1) (2011) 155–161.
- [17] A. Sierou, J. F. Brady, Accelerated stokesian dynamics simulations, *Journal of fluid mechanics* 448 (2001) 115–146.
- 845 [18] A. M. Fiore, J. W. Swan, Fast stokesian dynamics, *Journal of Fluid Mechanics* 878 (2019) 544–597.
- [19] L. Greengard, V. Rokhlin, A fast algorithm for particle simulations, *Journal of computational physics* 73 (2) (1987) 325–348.
- [20] B. Siddani, S. Balachandrar, Point-particle drag, lift, and torque closure models using machine learning: Hierarchical approach and interpretability, *Physical Review Fluids* 8 (1) (2023) 014303.
- [21] Z. Ma, Z. Ye, E. Safdarian, W. Pan, H-hignn: A scalable graph neural network framework with hierarchical matrix acceleration for simulation of large-scale particulate suspensions, Available at SSRN 5212995 (2025).
- 850 [22] V. Anand, V. Narsimhan, Sedimentation of spheroids in newtonian fluids with spatially varying viscosity, *Journal of Fluid Mechanics* 983 (2024) A28.
- [23] M. Reichert, H. Stark, Hydrodynamic coupling of two rotating spheres trapped in harmonic potentials, *Physical Review E* 69 (3) (2004) 031407.
- 855 [24] A. K. Townsend, The mechanics of suspensions, Ph.D. thesis, UCL (University College London) (2017).
- [25] B. Palmer, H. M. Aktulga, T. Gao, Smooth-rigid-body contact as a relcp: A recursively generated linear complementarity problem (2026). *arXiv:2506.14097*. URL <https://arxiv.org/abs/2506.14097>
- [26] T. Goldfriend, H. Diamant, T. A. Witten, Hydrodynamic interactions between two forced objects of arbitrary shape. i. effect on alignment, *Physics of Fluids* 27 (12) (2015).
- 860 [27] J. Barnes, P. Hut, A hierarchical o (n log n) force-calculation algorithm, *nature* 324 (6096) (1986) 446–449.
- [28] S. Kim, S. J. Karrila, *Microhydrodynamics: principles and selected applications*, Butterworth-Heinemann, 2013.
- [29] S. Batzner, A. Musaelian, L. Sun, M. Geiger, J. P. Mailoa, M. Kornbluth, N. Molinari, T. E. Smidt, B. Kozinsky, E (3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials, *Nature communications* 13 (1) (2022) 2453.
- 865 [30] T. Müller, A. Evans, C. Schied, A. Keller, Instant neural graphics primitives with a multiresolution hash encoding, *ACM Transactions on Graphics* 41 (4) (2022) 102:1–102:15.
- [31] W. Yan, R. Blackwell, Kernel aggregated fast multipole method: Efficient summation of laplace and stokes kernel functions, *Advances in Computational Mathematics* 47 (5) (2021) 69.
- [32] D. Malhotra, G. Biros, Pvfmm: A parallel kernel independent fmm for particle and volume potentials, *Communications in Computational Physics* 18 (3) (2015) 808–830.
- 870 [33] L. Wilson, N. Vaughn, R. Krasny, A gpu-accelerated fast multipole method based on barycentric lagrange interpolation and dual tree traversal, *Computer Physics Communications* 265 (2021) 108017.
- [34] R. Yokota, An fmm based on dual tree traversal for many-core architectures, *Journal of Algorithms & Computational Technology* 7 (3) (2013) 301–324.
- 875 [35] M. Burtcher, K. Pingali, An efficient cuda implementation of the treecode algorithm, *GPU computing gems Emerald edition* 6 (2011) 75–92.
- [36] T. Karras, Maximizing parallelism in the construction of bvhs, octrees, and k-d trees, in: *Proceedings of the Fourth ACM SIGGRAPH/Eurographics conference on High-Performance Graphics*, 2012, pp. 33–37.
- [37] H. J. Wilson, Stokes flow past three spheres, *Journal of Computational Physics* 245 (2013) 302–316.
- [38] B. Metzger, M. Nicolas, É. Guazzelli, Falling clouds of particles in viscous fluids, *Journal of Fluid Mechanics* 580 (2007) 283–301.
- 880 [39] K. W. Torre, R. D. Schram, J. de Graaf, Python-jax-based fast stokesian dynamics, *SciPost Physics Codebases* (2025) 056.
- [40] J. Liu, M. Robson, T. Quinn, M. Kulkarni, Efficient gpu tree walks for effective distributed n-body simulations, in: *Proceedings of the ACM International Conference on Supercomputing*, 2019, pp. 24–34.
- [41] D. Malhotra, G. Biros, Algorithm 967: A distributed-memory fast multipole method for volume potentials, *ACM Transactions on Mathematical Software (TOMS)* 43 (2) (2016) 1–27.
- 885 [42] M. Laudato, Neural-network closures for complex-shaped particles in the force-coupling method, *Computers & Fluids* (2026) 107101.